

1 Introducción aos scripts en PowerShell

1.1 Introducción

Ao igual que os scripts de bash, os de PowerShell tamén poden facer uso dunha ampla colección de elementos da linguaxe. Entre outros temos:

- Comentarios.
- Invocacións a comandos de Windows PowerShell.
- Invocacións a programas externos.
- Invocacións a elementos de .NET Framework (en Windows PowerShell) ou .NET Core (en PowerShell Core).
- Variables creadas no propio script.
- As variables de contorno do usuario que os executa.
- Os parámetros que se lle pasen durante a súa invocación
- Sentenzas condicionais
- Bucles
- Expresións regulares

1.2 Tipos de arquivos de script

En PowerShell, distinguimos 3 tipos de arquivos de script:

- Arquivos de script: .ps1
- Arquivos de datos de script: .psd1
- Arquivos de módulo de script: .psm1

Na meirande parte dos casos, traballaremos con ficheiros do primeiro tipo.

1.3 Execución de scripts

Recordamos neste punto que, por defecto, por defecto, PowerShell non permite executar scripts sen asinar, mais esta política pódese cambiar a vontade. Para consultar o estado actual da política, empregamos o cmdlet *Get-ExecutionPolicy*. Despois, podemos establecela mediante o cmdlet *Set-ExecutionPolicy*, que pode levar un destes catro valores:

- Restricted: non se poden executar scripts.
- AllSigned: só se poden executar scripts asinados.
- RemoteSigned: só os scripts descargados necesitan estar asinados.
- Unrestricted: pódese executar calquera script.

Para a nosa práctica, pode resultar máis sinxelo escoller a última opción.

Unha vez salvado este escollo, para executar un script basta con escribir a súa ruta e o seu nome na liña de comandos:

```
.\script.ps1
```

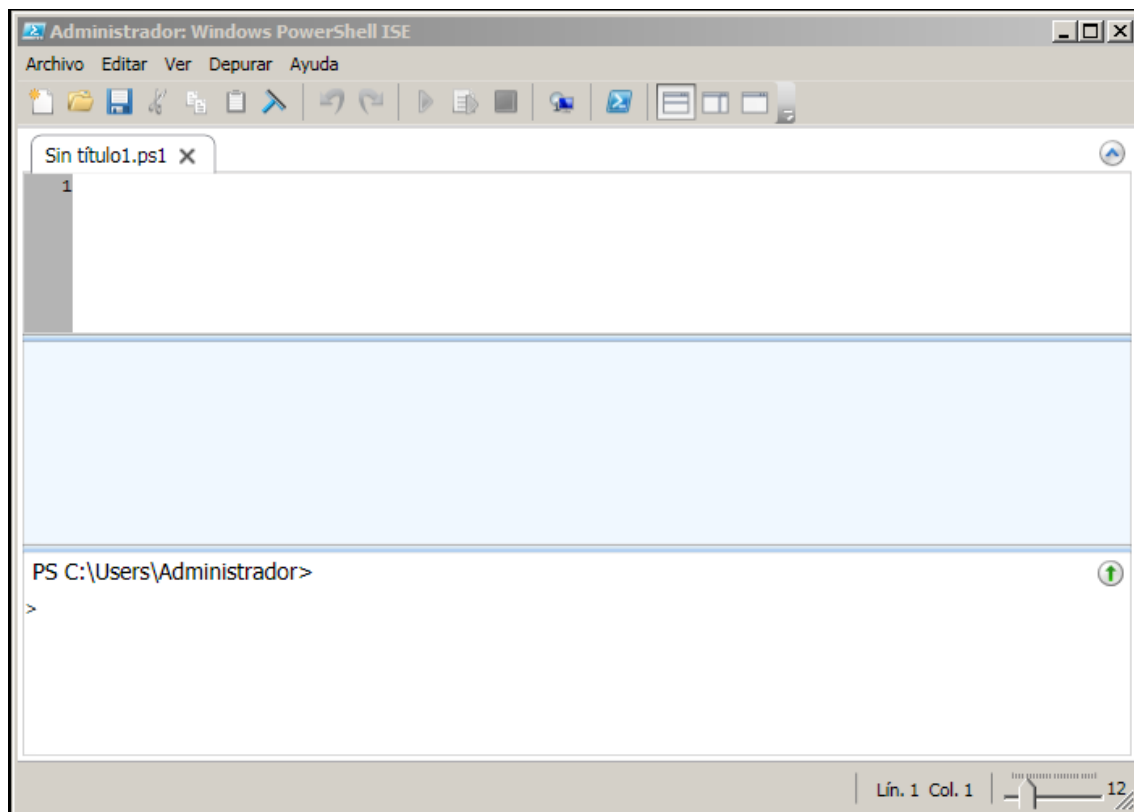
O executable de PowerShell tamén ten unha opción para pasarlle o script como parámetro:

Powershell[.exe] -File script.ps1

Neste caso, tamén temos un parámetro -ExecutionPolicy para establecer a política de execución nesa sesión, sen modificar a que hai no rexistro.

1.4 PowerShell ISE

PowerShell Integrated Scripting Environment (ISE) é unha ferramenta para crear, executar e depurar scripts. Aínda que xa se pode empregar desde a versión 2.0 de PowerShell, comeza a ser realmente potente a partir da 3.0, cando mellora as funcións de autocompletado e de listar os cmdlets dispoñibles. Este software só se pode empregar ata Windows PowerShell 5.1, versión que se considera “finalizada” e que polo tanto só recibirá o mantemento básico [1]. Pódese ver unha comparativa entre versións de ISE en [2].



O vello PowerShell ISE 2.0 en Windows 2008 R2

Desde PowerShell 2.0 pódense crear bloques de comentarios para non ter que escribir «#» en cada liña

Abrimos o bloque con <#

E pechámolo con #>

```
<#
```

```
    Isto é un bloque de comentario e bla bla bla
```

```
#>
```

1.6 Axuda baseada en comentarios

Isto que acabamos de ver é moi útil para escribir a axuda dun cmdlet ou script, xa que esta adóitase escribir como un comentario largo col formato:

```
.<NOME_SECCIÓN>
```

```
<CONTIDO>
```

Exemplos de seccións predefinidas son:

.SYNOPSIS – Breve descrición da función do cmdlet.

.DESCRIPTION – Descrición estendida do que fai o cmdlet.

.PARAMETER <NOMBRE_PARAM> - Información do parámetro. Meteremos un .PARAMETER por cada parámetro na función ou script.

.EXAMPLE – Exemplo de uso do comando. Hai que repetir esta palabra por cada exemplo.

.INPUTS – Os tipos dos obxectos que se pasan como entrada ao cmdlet, segundo os definidos en .NET Framework. Tamén é posible escribir unha descrición do parámetro.

.OUTPUTS – Os tipos dos obxectos que devolve o cmdlet, segundo os definidos en .NET Framework. Tamén é posible escribir unha descrición do parámetro.

.NOTES – Outras notas informativas sobre o funcionamento do script.

.LINK – O nome dun tema relacionado. Os contidos que poñamos aquí aparecerán na sección Related Links da axuda.

.COMPONENT – A tecnoloxía ou característica que usa (ou está relacionada con) a función ou o script. Esta información aparecerá cando o usuario invoque a axuda co parámetro -Component.

.ROLE – O rol de usuario na axuda. Esta información aparecerá cando o usuario invoque a axuda co parámetro -Role.

.FUNCTIONALITY – O uso para o que a función foi concebida. Esta información aparecerá cando o usuario invoque a axuda co parámetro -Functionality.

`.FORWARDHELPTARGETNAME <NOME_COMANDO>` - Redirixe á axuda do comando especificado. Pode ser a axuda dunha función, script, cmdlet ou provider.

`.FORWARDHELPCATEGORY <NOME_CATEGORÍA>` - Especifica a categoría da axuda do ítem no parámetro `.FORWARDHELPTARGETNAME`. Pode ser "Alias", "Cmdlet", "HelpFile", "Function", "Provider", "General", "FAQ", "Glossary", "ScriptCommand", "ExternalScript", "Filter" ou "All". É imprescindible usalo cando hai varios ítems co mesmo nome.

`.REMOTEHELPRUNSPACE <VARIABLE_PSSESSION>` - Especifica a sesión que contén o tema de axuda. Esta información úsase no cmdlet `Export-PSSession` para atopar a axuda dos comandos exportados.

`.EXTERNALHELP <FICHEIRO_AXUDA_XML>` - Especifica a ruta e/ou o nome dun arquivo de axuda en formato XML para o script ou función.

Toda esta axuda pode colocarse en diversos lugares, dependendo de se é para un script ou para unha función:

- Se é para un script, colocase ao comezo do mesmo, e pode ir separada do código por unha ou varias liñas en branco.
- Se é para unha función, colocase xusto antes da función, sen ir separada da cabeceira por liñas en branco. Tamén pode colocarse ao comezo do corpo ou ao final do mesmo.

Vexamos entón un exemplo de axuda baseada en comentarios:

```
<#  
  
.SYNOPSIS  
  
Calcula o factorial.  
  
.DESCRIPTION  
  
Esta función calcula o factorial dun número enteiro.  
  
.PARAMETER Numero  
  
Especifica o número do que imos calcular o factorial.  
  
.INPUTS  
  
Ningún. Non se poden pasar obxectos polo pipe.  
  
.OUTPUTS  
  
System.Int. Resultado de calcular o factorial  
  
.EXAMPLE  
  
PS> .\Factorial.ps1 -Numero 3  
  
#>
```

Na axuda de Microsoft hai máis exemplos deste tipo de axudas e de como redirixir á axuda doutro comando, a algún ficheiro externo, etc.

Referencias:

[1] "Microsoft. About Comment-based Help". En liña:

https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_comment_based_help. Accedido o 09-09-2019

1.7 Invocación de programas do sistema e built-ins

PowerShell incorpora moitos cmdlets por defecto para conseguir as funcionalidades básicas do sistema. Pódese ver rapidamente unha lista por categorías en [1], aínda que ao final deste tema ofreceremos unha lista máis actualizada.

Tamén se poden executar outros programas que se atopen no sistema, sempre e cando o usuario en cuestión teña permiso. Para facer iso, simplemente tecleamos a orde e os seus argumentos, se os houbera:

```
ORDE arg_1 arg_2 ... arg_n
```

Tamén é posible poñer varios comandos nunha liña separándoos por «;». Por exemplo:

```
Write-Host "Hola";Write-Host "Mundo"
```

O acento inverso (backtick, «`) serve para continuar un comando na seguinte liña. Para iso, necesitamos poñer un espazo antes do mesmo, mais non debe levar espazo despois:

```
Write-Host `
"Hola Mundo"
```

Para invocar comandos externos temos os operadores «.» e «&»

```
. meuArquivo.ps1
& meuArquivo.ps1
```

O carácter «.» mete as definicións dos comandos externos no script actual, mentres que «&» non o fai [2]. Este tema está relacionado cos ámbitos, que estudaremos máis adiante.

Vexamos un pequeno exemplo de script:

```
# Isto é un comentario
<#
    Isto é un comentario en bloque
#>
Write-Host "Isto é unha mensaxe sinxela con Write-Host"
```

```
Write-Host "E agora imos a usar o built-in de Get-Culture: "  
  
Get-Culture  
  
Write-Host "E finalmente imos a usar o comando dir:"  
  
& dir # e isto é outro comentario
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1  
  
Isto é unha mensaxe sinxela con Write-Host  
  
E agora imos a usar o built-in de Get-Culture:  
  
LCID           Name           DisplayName  
----           -  
3082           es-ES           Español (España)  
  
E finalmente imos a usar o comando dir:  
  
...  
  
PS C:\>
```

Referencias:

[1] Guy Thomas. "Windows PowerShell's Cmdlets (Built-In)". En liña: http://www.computerperformance.co.uk/powershell/powershell_cmdlets_builtin.htm.

Accedido o 09-09-2019.

[2] SS64.com. "Source or dot operator" En liña <https://ss64.com/ps/source.html>. Accedido o 09-09-2019.

1.8 Variables

En PowerShell pódense definir e usar variables dentro dun script de varias formas. A primeira é ao estilo de bash, referíndonos a ela mediante \$NOME_VARIABLE e asignando un valor con «=». Por exemplo:

```
$MENSAXE = "Hola"
```

Nótese que a diferenza de bash, en PowerShell o símbolo «\$» úsase incluso na declaración e nas asignacións. Pola súa parte, As asignacións tamén poden levar espazos ao redor do símbolo «=».

Ademais, usaremos \${NOME_VARIABLE} en lugar de \$NOME_VARIABLE se o nome ten caracteres non estándar.

A segunda forma de manipular variables é a través dos cmdlets *New-Variable*, *Get-Variable* e *Set-Variable*

```
New-Variable -Name "nome" -Value "Chuck" -Option ReadOnly  
Get-Variable -Include M*,P* -ValueOnly  
Set-Variable -Name "desc" -Value "descripción"
```

Para borrar una variable úsase:

```
Remove-Item variable:<VARIABLE>
```

Pero que vantaxe pode ter empregar esta maneira? Pois ben, os cmdlets *New-Variable*, *Get-Variable* e *Set-Variable* teñen moitas opcións de interese. Por exemplo, *New-Variable* ten, entre outras, opcións para crear variables de solo lectura (constantes) ou para crear variables privadas (non se poden alterar fóra da sesión) [1]. Pola súa parte, *Get-Variable* ten, entre outras, opcións para atopar todas as variables que respondan a un patrón e para obter o valor da variable xunto con outros metadatos da mesma [2]. Finalmente, *Set-Variable* ten algunhas opcións similares a *New-Variable* [3]. Ademais, como os outros cmdlets, pódense usar con opcións comúns como a de *-whatif*, *-confirm*, etc.

Por certo, agora que xa sabemos como crear variables, podemos probar a diferenza entre o «.» e o «&» que vimos anteriormente. Para iso, creamos este pequeno script:

```
$n = 1;&{$n = 2};$n  
$n = 1;.{ $n = 2 };$n
```

E o resultado da execución será:

```
PS C:\> .\script.ps1  
  
1  
  
2  
  
PS C:\>
```

Isto ocorre porque con «&» non se está a gardar as accións que ocorren no bloque de código entre chaves {}, que se pode considerar coma un script de seu, mentres que con «.» temos o comportamento contrario, o que sucede no bloque expórtase ao exterior.

Referencias:

[1] Microsoft. "New-Variable". En liña: <https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/new-variable>. Accedido o 10-09-2019.

[2] Microsoft. "Get-Variable". En liña: <https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-variable>. Accedido o 10-09-2019.

[3] Microsoft. "Set-Variable". En liña: <https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/set-variable>. Accedido o 10-09-2019.

1.9 Tipos

PowerShell é unha linguaxe orientada a obxectos e, polo tanto, as variables son obxectos do sistema. Ata agora non puidemos darnos conta porque, por defecto, a clase (tipo) de obxecto é asignada automaticamente. Por exemplo:

```
$var = 5
```

Interprétase como Int32 (número enteiro de 32 bits).

```
$var = "5"
```

Interprétase como String (cadea de texto).

Ademais, a clase pódese cambiar automaticamente en tempo de execución. Por exemplo, se facemos `$var = 5` asigna un enteiro, pero se logo facemos `$var = "Hola"`, este convértese a cadea de texto.

En calquera caso, pódese forzar a que unha variable sexa dunha certa clase. Para iso, simplemente temos que indicar o tipo entre corchetes antes da declaración:

```
[int32]$var = 5
```

Desta maneira, este tipo xa non pode cambiar en tempo de execución.

Pero entón, que tipos de variables temos dispoñibles? Pois ben, a cousa é que en Windows PowerShell pódese asignar calquera tipo definido en .NET Framework, e da mesma maneira en PowerShell Core pódese empregar calquera tipo definido en .NET Core. Como, por suposto, non podemos coñecerlos todos listaremos só os tipos máis comúns [1]:

Tipo	Significado
[string]	Cadea de caracteres Unicode de lonxitude fixa.
[char]	Un carácter Unicode de 16 bits.
[byte]	Un carácter de 8 bits sen signo.
[int]	Un enteiro de 32 bits con signo.
[long]	Un enteiro de 64 bits con signo.
[bool]	Booleano (valor True ou False).
[decimal]	Un valor decimal de 128 bits.
[single]	Un número de punto flotante de precisión sinxela (32 bits)
[double]	Un número de punto flotante de precisión dobre (64 bits)
[DateTime]	Data e hora
[xml]	Un obxecto en XML
[array]	Un array de valores
[hashtable]	Unha táboa hash
[wmi]	Instancia ou recopilación de Windows Management Instrumentation (WMI)
[wmiclass]	Clase WMI
[adsis]	Obxecto de Servizos de Active Directory

Referencias:

[1] SS64.com. "PowerShell Data Types" <https://ss64.com/ps/syntax-datatypes.html>. Accedido o 10-09-2019.

1.10 Membros

Recordemos que as variables en PowerShell son obxectos e, como tales, pódense manipular mediante os mecanismos que elas mesmas incorporan segundo o seu tipo. Por exemplo, se temos unha cadea de texto en \$var e queremos pasala a minúsculas non imos invocar ningún comando de Windows, senón que faremos:

```
$var.ToLower()
```

Coma en moitas linguaxes de programación orientada a obxectos, esta operación devolve outro String co seu contido en minúsculas, mais non modifica o valor da variable (obxecto) orixinal.

En xeral, en PowerShell non imos ter función intrínsecas de manipulación de datos. Non son necesarias, posto que os obxectos presentarán os métodos adecuados para realizar as mesmas operacións.

Para ver todas as funcionalidades dunha variable, executaremos o comando:

```
Get-Member -inputobject <VARIABLE>
```

Con PowerShell incluso poderíamos definir os nosos propios tipos de datos e as súas funcionalidades nos ficheiros de configuración de PowerShell. Sobre isto, podedes obter máis información en [1].

Vexamos un script de exemplo no que manipulamos algunhas variables:

```
$mensaxe = "Ola"

Write-Host "Mensaxe en maiúsculas:" $mensaxe.ToUpper()

Write-Host "Procuramos a posición dpa a:" $mensaxe.IndexOf('a')

Write-Host "Inserimos un u:" $mensaxe.Insert(1, "u")

$numero = 5

Write-Host "Comparo se é igual que 4:" $numero.Equals(4)

Remove-Variable numero

Write-Host "Número:" $numero
```

O resultado da execución será:

```
PS C:\> .\script.ps1

Mensaxe en maiúsculas: OLA

Buscamos la posición do a: 2
```

Inserimos un u: Ola

Comparo se é igual que 4: False

Número:

PS C:\>

Nota: Ao igual que *Remove-Variable numero* tamén debería de funcionar *Remove-Item variable:numero*, pero por algún motivo en PowerShell 5 da un erro cando executamos o script na consola (mais non o da cando executamos os comandos directamente na consola ou o script no PowerShell ISE, ou nalgúnhas versións antigas de PowerShell).

Referencias:

[1] Don Jones de Microsoft. "Windows PowerShell. El poder de las variables". En liña: <https://technet.microsoft.com/es-es/library/2007.03.powershell.aspx>. Accedido o 11-09-2019

1.11 Variables automáticas

PowerShell incorpora por defecto varias constantes e variables. Algunhas delas son:

Variable	Significado
\$true	Representa o valor booleano «verdadero».
\$false	Representa o valor booleano «falso».
\$^	Refírese ao primeiro termo do comando anterior.
\$\$	Refírese ao último termo do comando anterior.
\$?	Devolve \$false se o comando anterior rematou cun erro, \$true en caso contrario. Por algunha razón non colle todos os erros.
\$_ e \$PSItem	Almacena un valor temporal, que pode ser o parámetro dun comando que se está a procesar ou o erro que devolveu un comando. O alias \$PSItem só está dispoñible desde PS 3.0.

Por exemplo, se temos o seguinte comando:

```
Write-Host "Ola Mundo"
```

\$^ devolve «Write-Host»

\$\$ devolve «Ola Mundo»

En realidade, hai bastantes variables automáticas máis, pero o resto irémolo vendo pouco a pouco cando sexa necesario. Se precisades toda a información sobre variables automáticas basta con executar o cmdlet:

```
Get-Help About_Automatic_Variables
```

Ou visitar a web de Microsoft equivalente [1].

Referencias:

[1]. Microsoft. "About automatic variables" En liña: https://docs.microsoft.com/es-es/powershell/module/microsoft.powershell.core/about/about_automatic_variables. Accedido o 10-09-2019.

1.12 Mais sobre clases e obxectos

En PowerShell, todos os «obxectos» pertencen a unha «clase» (tipo). A clase, ven sendo unha especie de «esquema» que define que «membros» (atributos e métodos) soportan os obxectos pertencentes a ela.

Para acceder a un membro dun obxecto, empregamos o operador «.». Por exemplo:

```
$minhaCadea.ToLower()
```

Accede á operación (método) "ToLower()" do obxecto \$minhaCadea.

Como en moitas linguaxes de POO, as clases tamén poden ter os seus propios «membros». A estes atributos e métodos se lles denomina «estáticos».

Para acceder a un membro estático dunha clase empregamos «::». Por exemplo:

```
[System.Console]::Beep()
```

Accede á operación "Beep()" da clase System.Console, non dun obxecto de tipo System.Console.

Antes vimos cómo acceder aos membros dun obxecto, mais para ver os membros estáticos dunha clase teremos que facer:

```
Get-Member -InputObject <CLASE> -static
```

Por exemplo:

```
Get-Member -InputObject System.Console -static
```

É preciso que comparedes o resultado co de non empregar o modificador "-static".

```
Get-Member -InputObject <CLASE>
```

Por exemplo:

```
Get-Member -InputObject System.Console
```

Ademais disto, PowerShell incorpora unha serie de operadores para comprobar o tipo dun obxecto:

Operador	Significado
-is	Comproba se un obxecto é de certo tipo
-isnot	Comproba se un obxecto non é dun certo tipo
-as	Converte un obxecto a un certo tipo. Non devolve erro en caso de fallo

Imos ver agora un pequeno script de exemplo de uso destes operadores:

```

$a = 1
$b = "1"
$a -is [int]
$a -is $b.GetType()
$a -isnot $b.GetType()
$b -isnot [int]
($b -as [int]) -is $b.GetType()
($b -as [int]) -is $a.GetType()

```

A execución debería dar o seguinte resultado:

```

PS C:\> .\script.ps1
True
False
True
True
False
True
PS C:\>

```

Sobre todo este tema tedes máis información na web de Microsoft [1]. Ademais, desde PowerShell 5.0 un pode definir a súas propias clases [2].

Referencias:

[1] Microsoft. "About classes". En liña: https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_classes. Accedido o 14-09-2019.

[2] Dr. Scripto. "Powershell 5: Create simple class". <https://blogs.technet.microsoft.com/heyscriptingguy/2015/09/01/powershell-5-create-simple-class/>. Accedido o 12-09-2019.

1.13 Avaliar sub-expresións

En ocasións queremos que unha certa sub-expresión se avalíe antes que a expresión que a contén. Por exemplo, no seguinte caso queremos que `$var.length` se avalíe antes que a cadea de texto.

```
Write-Host "Hai $var.length letras"
```

Pódense avaliar sub-expresións meténdoas dentro dos símbolos `$( )`. Por exemplo:

```
$ (3+2)
```

```
$ ($name.length)
```

```
Write-Host "Hai $($var.length) letras"
```

Incluso é posible meter varias instrucións no operador `$()`, separadas por «;»

Vexamos agora un script de proba:

```
$mensaxe = "Ola"
```

```
Write-Host "Mensaxe En maiúsculas:" $mensaxe.ToUpper()
```

```
Write-Host "Mensaxe En maiúsculas: $($mensaxe.ToUpper())"
```

```
Write-Host "Isto son 3 + 2: $(3+2)"
```

```
$(Get-WMIObject win32_bios) # Obtén os datos de la BIOS
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
Mensaxe En maiúsculas: OLA
```

```
Mensaxe En maiúsculas: OLA
```

```
Esto son 3 + 2: 5
```

```
SMBIOSBIOSVersion : VirtualBox
```

```
Manufacturer      : innotek GmbH
```

```
Name              : Default System BIOS
```

```
SerialNumber       : 0
```

```
Version           : VBOX - 1
```

```
...
```

```
PS C:\>
```

1.14 Operacións aritméticas e lóxicas

As sub-expresións vistas no apartado anterior, entre outras cousas, emprégase para a avaliación de expresións aritméticas e lóxicas, En principio PowerShell dispón dos operadores que aparecen na seguinte táboa:

Operador	Significado
=, +=, -=, *=, /=, %=	Asignacións
VAR++ y VAR--	Post-incremento de variable
++VAR y --VAR	Pre-incremento de variable
+, -, *, / y %	Suma y resta / más y menos unarios, multiplicación, división y módulo (resto)

-bnot, -band, -bor y -bxor	Negación, AND, OR y XOR bit a bit
-shl, -shr	Bits desplazados a la izquierda/derecha un cierto número de veces conservando el signo [PS 3.0+]
-not (!), -and, -or y -xor	Operaciones lógicas: negación, AND, OR y XOR
-eq, -ne, -gt, -ge, -lt, -le	Igualdad, desigualdad, mayor que, mayor o igual que, menor que, menor o igual que

Moitas outras operacións matemáticas realízanse empregando a librería Math de .NET [1]. Pódese ver una lista de métodos mediante o comando:

```
[system.math] | gm -static
```

Ademais, podedes ver outros operadores de comparación en [2].

Referencias:

[1] "Math in PowerShell": <http://www.madwithpowershell.com/2013/10/math-in-powershell.html>. Accedido o 12-09-2019.

[2] SS64.com "Comparison operators". En liña: <https://ss64.com/ps/syntax-compare.html>. Accedido o 12-09-2019

1.15 Cadeas de texto

Los operadores -eq, -ne, -gt, -lt, etc. están sobrecargados e tamén funcionan con cadeas de texto. Os operadores -eq e -ne comprobam a igualdade e a desigualdade respectivamente, mentres que -gt, -ge, -lt e -le miran a orde lexicográfica.

PowerShell tamén dispón dos operadores -like e -notlike, que serven para comprobar se unha cadea segue ou non un patrón no que se poden incluír comodíns (p. ex.: «*»). Temos así que

```
"Rafa" -like "R*"
```

Devolve "True".

Todos estes operadores son insensibles a maiúsculas e teñen unha versión sensible a maiúsculas poñéndolles unha «c» entre o símbolo «-» e o nome. Tamén se pode poñer unha «i» entre o «-» e o nome para remarcar que é insensible a maiúsculas. Así por exemplo:

```
"Rafa" -ceq "rafa"
```

Devolve "False", mentres que:

```
"Rafa" -ieq "rafa"
```

Devolve "True".

Podemos probar os seguintes exemplos:

Expresión	Resultado
"Rafa" -eq "Roberto"	False
"Rafa" -eq "rafa"	True

"Rafa" -ceq "rafa"	False
"Rafa" -ieq "rafa"	True
"Rafa" -ge "Roberto"	False
"Rafa" -ge "rafa"	True
"Rafa" -cge "rafa"	True
"Rafa" -ige "rafa"	True
"rafa" -ge "Rafa"	True
"rafa" -cge "Rafa"	False
"rafa" -ige "Rafa"	True
"Rafa" -like "r*"	True
"Rafa" -clike "r*"	False
"Rafa" -ne "Roberto"	True
"Rafa" -ne "rafa"	False
"Rafa" -cne "rafa"	True
"Rafa" -ine "rafa"	False
"Rafa" -notlike "r*"	False
"Rafa" -cnotlike "r*"	True

1.16 Arrays

Un array é unha colección ordeada de obxectos. En PowerShell hai varias formas de definilos. A primeira, é separando os obxectos por comas. Por exemplo:

```
$meuArray = 1, "Ola", 3.5, "Mundo"
```

A segunda é usando a notación explícita "@()". Por exemplo:

```
$meuArray = @(1, "Hola", 3.5, "Mundo")
```

A terceira é crear un array baleiro con:

```
$meuArray = @()
```

Ademais, pódense distribuír os valores dun array en variables:

```
$var1, $var2, $var3, $var4 = $meuArray
```

Tamén se poden definir rangos de números de forma sinxela. Por exemplo, facer:

```
$meuArray = (1..7)
```

É o mesmo que facer

```
$meuArray = 1, 2, 3, 4, 5, 6, 7
```

Para crear arrays de varias dimensións, se utilizan parénteses:

```
$meuMultiArray = @((1,2,3),(40,50,60))
```

Tamén se pode dar tipado forte a un array:

```
[int[]] $meuArray = 12, 64, 8, 64, 12
```

Podemos capturar un valor dun array con:


```
$meuArray[<POSICIÓN>]
```

E se empregamos números negativos se comeza polo final.

Tamén podemos capturar un rango con:

```
$meuArray[<INICIO>..<FIN>]
```

E para varias dimensións:

```
$meuArray[<POS1>][<POS2>]...
```

Incluso se poden capturar valores individuais mesturados con rangos. Os valores individuais sepáranse do rango cun signo «+». Por exemplo:

```
$meuArray[1,2+4..9]
```

Captura os elementos 1, 2 e do 4 ao 9.

Unha propiedade que teñen os arrays de PowerShell é que son inmutables. Por tanto, para engadir un elemento a un array, PowerShell crea un novo array co elemento xa engadido e desbota o array vello. Para engadir un elemento a un array úsase o operador «+=». P. ex.:

```
$países += 'España'
```

A suma de dous arrays concatena os arrays

```
$impares = @(1,3,5,7,9)
```

```
$pares = @(2,4,6,8,10,12)
```

```
$todos = $impares + $pares
```

Neste caso \$todos sería @(1,3,5,7,9,2,4,6,8,10,12)

Para saber se unha colección contén un elemento podemos usar os operadores de coleccións -contains, -notcontains, -in e -notin. Engadindo «c» diante do nome fai as comparacións sensibles a maiúsculas (P. ex.: «-ccontains»).

Operador	Significado
-contains	Contido nunha colección
-notcontains	Non contido nunha colección
-in	Coma -contains pero cos operadores ao revés [PS 3.0+]
-notin	Coma -notcontains pero cos operadores ao revés [PS 3.0+]

Por exemplo:

```
$unArray = @("ola","adeus","veña")
```

```
Write-Host $($unArray -contains "ola")
```

```
Write-Host $($unArray -contains "grazas")
```

```
Write-Host $($unArray -ccontains "OLA")
```

Devolve:

```
True
```

```
False
```

```
False
```

Outra peculiaridade dos arrays é que se pode establecer filtros con -eq, -gt, -lt.... Por exemplo, para obter os elementos dun array que son iguais que 2 fariamos:

```
$a = @(1,2,3,2)
```

```
$a -eq 2
```

E devolvería

```
2
```

```
2
```

Tede coidado ao comprobar que un array non é nulo:

```
$a -eq $null # falla, pois retorna $null
```

```
$null -eq $a # funciona, pois retorna $false
```

Para obter máis información sobre un array podemos empregar varias operacións. A primeira é a que amosa a clase do array (o tipo de datos):

```
$meuArray.GetType()
```

A segunda, amosa as propiedades e as operacións do array:

```
Get-Member -inputobject $miArray
```

A terceira, é a que amosa as propiedades dos elementos do array:

```
$meuArray | Get-Member
```

Nótese que a expresión:

```
Get-member $meuArray
```

Devolve un erro.

Vexamos un exemplo concreto de script:

```
[Int32[]]$meuArray = @(2,3,4)
```

```
$meuArray[1] = 9
```

```
Write-Host "Valor de [1]: $($meuArray[1])"
```

```
Write-Host "Tipo do array: $($meuArray.gettype())"

Write-Host "Get-Member con -inputobject"

Get-member -inputobject $meuArray

Write-Host "Get-Member con pipe"

$meuArray | Get-Member

Get-Member $meuArray
```

A execución daría o seguinte resultado:

```
PS C:\> .\script.ps1

Valor de [1]: 9

Tipo do array: System.Int32[]

Get-Member con -inputobject

... (Aquí se amosan os membros de $meuArray)

Get-Member con pipe

... (Aquí se amosan os membros dos elementos de $meuArray)

Get-Member : No se ha especificado ningún objeto para el cmdlet
get-member.

PS C:\>
```

Máis información sobre arrays en [1].

Referencias:

[1] SS64.com. "Arrays". En liña: <https://ss64.com/ps/syntax-arrays.html>. Accedido o 12-09-2019.

1.17 Táboas hash

Unha táboa hash es unha colección de valores que se poden procurar por unha clave. A procura é moito máis rápida que nun array. En PowerShell pódese crear una táboa hash mediante o construto @{}. Ademais, pódense engadir elementos durante la declaración poñendo dentro das llaves elementos <CLAVE>=<VALOR>; (nótese que o ";" é esencial para separar varias asignacións. Por suposto, tamén se poden engadir elementos despois usando unha expresión de tipo \$var[<CLAVE>] = <VALOR>. Finalmente, pódese acceder aos elementos dunha táboa hash usando \$var[<CLAVE>].

Por exemplo:

```
$libros = @{

"978-84-204-7248-5" = "O mestre de esgrima";
```

```

"978-84-204-7269-0" = "A táboa de Flandes";
"978-84-204-8353-5" = "O Capitán Alatriste";
"978-84-204-1309-9" = "O tango da Gardia Vella";
}

Write-Host "Esquecíase un..."

$libros["978-84-204-2894-9"] = "Falcó";

Write-Host "O libro ...2894-9 é $($libros['978-84-204-2894-9'])"

```

E a execución dá o seguinte resultado:

```

PS C:\> .\script.ps1

O libro ...2894-9 é Falcó

PS C:\>

```

1.18 Parameter splatting

Desde PowerShell 2.0, é posible usar os pares (clave, valor) dunha táboa hash como parámetros dun comando. Primeiro se declara unha táboa hash que conteña os nomes dos parámetros e os seus valores. Por exemplo,

```
$parms = @{...}
```

E despois, pasamos esa táboa como parámetro do comando con «@» en lugar de «\$». Por exemplo:

```
Get-WmiObject @parms
```

Vexamos un script de exemplo:

Creamos un hash cos parámetros do cmdlet New-Variable

```

$meuHash = @{
    "Name" = "minhaNovaVariable";
    "Value" = "Seu valor";
    "Description" = "Unha variable de exemplo";
    "Option" = "ReadOnly";
    "Visibility" = "Public";
}

```

E pasámoslle o hash á función New-Variable como parámetro

```
New-Variable @meuHash
```

```
Write-Host "Miña nova variable: $($minhaNovaVariable)"  
  
$minhaNovaVariable = 3
```

E a execución daría o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
A miña nova variable: Seu valor
```

No se puede sobrescribir la variable `miNuevaVariable` porque es de solo lectura o constante.

...

```
PS C:\>
```

Más información en [1].

Referencias:

[1] Don Jones, de Microsoft. "Windows PowerShell: splatting". En liña: <https://technet.microsoft.com/en-us/library/gg675931.aspx>. Accedido: 13-09-2019.

1.19 Argumentos

Como é natural, pódense pasar argumentos a un script cando o invocamos. Por exemplo:

```
.\script.ps1 -val 12
```

Unha forma de acceder a eles é polo seu nome mediante o comando *param*. Por exemplo:

```
param([Int32]$val=30)
```

Desta maneira, accederíamos ao argumento que pasamos con «-val» na liña de comandos, asignándolle o valor 30 se non se lle pasou ningún.

Unha limitación que ten o comando *param* é que ten que ser a primeira orde do script ou función no que o empreguemos. Ademais, as cadeas de texto deben ir con comiñas simples. Finalmente, se hai varios argumentos, estes deben ir separados por comas.

Tamén se poden pasar comandos como valor do parámetro. Por exemplo, no *param* poderíamos meter unha expresión coma a seguinte:

```
[string]$password = $( Read-Host -asSecureString "Input  
password" )
```

Os parámetros sen valor (flags) pódense recoller como un tipo *switch*:

```
[switch]$val = $false
```

Os argumentos tamén poden ter atributos especiais. Estes póñense entre corchetes antes de declarar o tipo. Cada un destes atributos indica unha característica distinta, por exemplo, a obrigatoriedade de pasar o parámetro, se se pode pasar por pipeline, etc. Por exemplo:

```
[Parameter(Mandatory = $true, ValueFromPipeline = $true, ValueFromPipelineByPropertyName = $true)][string] parametro
```

Desde PowerShell 3.0, se non se pon o valor por defecto dun argumento, asúmese que é `$true`. Porén, en PowerShell 1.0 e 2.0, non poñelo fai que non se poida executar o script.

Outra forma de acceder aos argumentos é por posición. O array especial `$args` contén a lista de parámetros, de forma que para acceder ao primeiro argumento se emprega `$args[0]`, e así sucesivamente. Unha vantaxe deste mecanismo é que se pode acceder aos argumentos en calquera lugar do script.

Para coñecer o número de parámetros que recibiu o script, empregamos as expresións `$args.length` ou `$args.count`

Unha limitación de PowerShell é que non se poden combinar ambas formas de acceder aos parámetros. Se se usa o comando `params`, en automático a variable `$args` estará baleira.

Vexamos agora un par de exemplos de scripts en acción. O primeiro:

```
param([Int32]$val=30,  
[Int32]$val2=25,  
[string]$val3='ola',  
[string]$val4='ti')  
Write-Host "Val: " $val  
Write-Host "Val2: " $val2  
Write-Host "Val3: " $val3  
Write-Host "Val4: " $val4
```

E a execución daría o seguinte resultado:

```
PS C:\> .\script -val 12 -val3 adeus  
Val: 12  
Val2: 25  
Val3: adeus  
Val4: ti  
PS C:\>
```

E agora, a segunda forma de empregar os argumentos:

```
Write-Host "args: " $args  
Write-Host "tamaño 1: " $args.Count
```

```
Write-Host "tamaño 2: " $args.Length  
Write-Host "param 0: " $args[0]  
Write-Host "param 1: " $args[1]
```

E unha execución daría un resultado coma este:

```
PS C:\> .\script -val 12 -val3 adeus  
  
args:  -val 12 -val3 adeus  
  
tamaño 1:  4  
tamaño 2:  4  
  
param 0:  -val  
param 1:  12  
  
PS C:\>
```

Más información sobre parámetros en [1].

Referencias:

[1] SS64.com. "Parameters". En liña: <https://ss64.com/ps/syntax-args.html>. Accedido o 13-09-2019.

1.20 Expresións regulares

Unha expresión regular é un patrón que representa unha serie de cadeas de texto. PowerShell soporta os seguintes elementos en expresións regulares:

- «*» representa calquera cadea.
- «.» representa calquera carácter.
- [abc] pode representar unha letra «a», «b» ou «c».
- [a-z] pode representar unha letra no rango do «a» ao «z».
- [^abc] pode representar unha letra que non sexa «a», «b» ou «c».
- [^a-f] pode representar unha letra que non estea no rango da «a» ao «f».
- «?» representa cero ou máis instancias do carácter anterior.
- «+» representa unha ou máis instancias do carácter anterior.
- «^» procura na que a cadea ten que comezar co patrón indicado a continuación.
- «\$» procura na que a cadea ten que rematar co patrón indicado xusto antes.

Ademais disto, PowerShell soporta as chamadas "clases", que non son máis que conxuntos de caracteres. Para representar calquera carácter dunha clase facemos:

```
\p{CLASE}
```

Ao final deste apartado ofrécese unha táboa coas clases existentes.

Outro operador soportado é o que representa expresións nas que o carácter pasado como parámetro está repetido N veces.

<CARÁCTER>{<N>}

Por exemplo:

s{5}

Representa a secuencia 'sssss'.

Se queremos representar as cadeas nas que o carácter estea repetido polo menos N veces, facemos:

<CARÁCTER>{<N> , }

Mentres que se queremos que o carácter estea repetido entre N e M veces, facemos:

<CARÁCTER>{<N,M>}

Por defecto, PowerShell procura a secuencia de caracteres mais longa que concorde co patrón que se lle pasa. Para procurar a secuencia máis curta, basta con engadir "?" ao final das expresións (p.ex.: «*?», «+?» , «{n}?», etc.).

Agora ben, como se comproba que unha cadea pertence a un determinado patrón? Pois para iso PowerShell déixanos empregar varios operadores. Para comezar, temos os operadores opostos -match e -notmatch que serven para buscar patróns que concordan cunha expresión regular. Por exemplo:

```
'abcde' -match '[xyz]+'
```

Devolve False porque a cadea non é unha secuencia de caracteres "x", "y" ou "z", e dentro da cadea tampouco podemos atopar ningunha subcadea que concorde con dito patrón. Porén:

```
'abcde' -match '[xyz]*'
```

Devolve True porque * implica 0 ou máis, e si hai unha subcadea de 'abcde', que é a subcadea baleira que fai match coa cadea [xyz]* cando o * vale exactamente "0 veces" (cadea baleira).

En xeral, para ver cales son as cadeas que concordan co patrón, o que podemos facer é imprimir unha táboa hash automática chamada \$matches que se xera cada vez que un match devolve True. Por exemplo:

```
PS C:\ > 'axxb' -match '[xyz]+'
```

```
True
```

```
PS C:\ > $matches
```

Name	Value
----	-----

Moito ollo! Esta variable especial non se limpa se logo facemos un match que devolve false!

O operador `-cmatch` é a versión sensible a maiúsculas de `match`, e o operador `-imatch` é un alias de `-match` (insensible a maiúsculas).

Por outra banda, o operador `-replace` serve para cambiar unha expresión regular por unha cadea noutra cadea. Por exemplo:

```
'abcde' -replace 'c',''
```

Devolve «abde», porque cambiamos o “c” polo baleiro.

De maneira similar, o operador `-creplace` é a versión sensible a maiúsculas e operador `-ireplace` é un alias de `-replace` (insensible a maiúsculas).

Nótese que no comando `-replace` tamén se pode trocar unha subcadea por esa mesma subcadea e algo máis usando «`$&`». Por exemplo:

```
'ABCD' -replace "[BC]", '$&x'
```

Troca cada «B» ou «C» por ese mesmo «B» o «C» máis un «x», resultando en «ABxCxD».

Tamén se poden asignar números ás partes reemplazadas creando «grupos de captura». Para crear un grupo de captura, se usan os parénteses «`()`». Por exemplo:

```
'ABCD' -replace "([AC])(.)", '$2-$1'
```

Chama \$1 a calquera concordancia (match) de “[AC]” e \$2 a calquera concordancia de “.”, e troca o patrón atopado polo capturado en \$2, un guión e o capturado en \$1, resultando en “B-AD-C”.

Incluso pódenselle poñer nomes a os grupos de captura con `?<NOME>` tras abrir parénteses, e logo se poden usar con `${NOME}`. Por exemplo:

```
'ABCD' -replace "(?<foo>[AC])(?<bar>.)", '${bar}-${foo}'
```

Usa “`${foo}`” e “`${bar}`” en lugar de \$1 y \$2, resultando “B-AD-C”.

Finalmente, imos practicar o uso de expresións regulares cun script sinxelo:

```
'Rafa' -match 'Ra[a-z]a'
```

```
'Rafa' -match 'Ra[^fgh]a'
```

```
'AaaCcc' -match 'A+C'
```

```
'Rafa' -match '\w+'
```

```
'Rafa' -match '\W+'
```

```
'00000000' -replace '0{5,}?','x'
```

```
'non tedes que ir á recuperación' -replace '^non ',''  
'non tedes que ir á recuperación' -replace '(ir)','v$1'
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1  
  
True  
  
False  
  
True  
  
True  
  
False  
  
x000  
  
tedes que ir á recuperación  
non tedes que vir á la recuperación  
  
PS C:\>
```

Como exercicio: probade tamén o sexto comando sen a interrogación, con 0{5,} a secas e vede a diferenza no resultado.

Podedes atopar máis información sobre expresións regulares en [1], [2] e [3].

Referencias:

[1] SS64.com “Regular expressions”. En liña: <https://ss64.com/ps/syntax-regex.html>. Accedido o 12-09-2019.

[2] Microsoft. “About regular expressions”. En liña: https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_regular_expressions. Accedido o 18-03-2020.

[3] Microsoft. “Character classes in regular expressions”. En liña: <https://docs.microsoft.com/en-us/dotnet/standard/base-types/character-classes-in-regular-expressions>. Accedido o 12-09-2019.

Anexo: táboa de clases soportadas por PowerShell:

Clase	Caracteres que engloba
\w	Letras e números, incluídas letras como á, å, ä, æ, ç, è, π, Ε, ψ, etc.
\W	Caracteres que non estean en \w
\s	Caracteres de espazo como [\f\n\r\t\v]
\S	Caracteres que non estean en \s
\d	Caracteres de díxitos
\D	Caracteres que non sexan díxitos
\p{Ll}	Letras maiúsculas

\p{Lu}	Letras minúsculas
\p{Lt}	Letras de título
\p{Lm}	Modificadores
\p{Lo}	Outras letras
\p{L}	Letras: unión de Ll, Lu, Lt, Lm e Lo
\p{Mn}	Marca, non espazado
\p{Mc}	Marcas, combinación de espazos
\p{Me}	Marcas, para rodear
\p{M}	Todas as marcas diacríticas, inclúe Mn, Mc y Me
\p{Nd}	Números, díxitos decimais
\p{NI}	Números, letras
\p{No}	Números, outros
\p{N}	Todos os números, inclúe Nd, NI e No
\p{Pc}	Puntuación, conector
\p{Pd}	Puntuación, raia
\p{Ps}	Puntuación, apertura
\p{Pe}	Puntuación, peche
\p{Pi}	Puntuación, comiñas iniciais (Pode comportarse como Ps ou Pe)
\p{Pf}	Puntuación, comiñas finais (Pode comportarse como Ps ou Pe)
\p{Po}	Puntuación, outras
\p{P}	Todos os caráct. de puntuación, inclúe Pc, Pd, Ps, Pe, Pi, Pf, e Po
\p{Sm}	Símbolo, matemáticas
\p{Sc}	Símbolo, moeda
\p{Sk}	Símbolo, modificador
\p{So}	Símbolo, outro
\p{S}	Todos los símbolos, inclúe Sm, Sc, Sk, e So
\p{Zs}	Separador, espazo
\p{Zl}	Separador, liña
\p{Zp}	Separador, parágrafo
\p{Z}	Todos los separadores, inclúe Zs, Zl e Zp
\p{Cc}	Outros, control
\p{Cf}	Outros, formato
\p{Cs}	Outros, substituto
\p{Co}	Outros, uso privado
\p{Cn}	Outros, non asignado
\p{C}	Todos os caracteres de control, inclúe Cc, Cf, Cs, Co, e Cn

Ademais, hai outros nomes de bloques soportados para certos tipos de alfabetos, símbolos especiais, marcas, etc. Por exemplo `\p{IsGreek}` representaría todas as letras gregas (Unicode 0370 - 03FF) e `\p{IsHiragana}` representaría o silabario *hiragana* do xaponés (Unicode 3040 - 309F). Podedes ver unha lista completa na sección «Supported Named Blocks» de [2].

1.2.1 Interacción co usuario

Como xa vimos con anterioridade, o cmdlet para escribir por pantalla é `Write-Host`. Ao igual que en `bash`, en ocasións será necesario escapar certos caracteres para construír outros caracteres especiais ou para desfacer ambigüidades. En calquera caso, é necesario saber que en PowerShell o carácter de escape e o acento inverso ou backtick «```».

A seguinte táboa amosa exemplos de caracteres especiais soportados en PowerShell:

Símbolo	Significado
`\$	Inserta un símbolo do dólar na cadea
`0	Inserta un valor nulo na cadea. Pódese usar \$null no seu lugar
`a	Alerta (fai un son de beep)
`b	Backspace
`f	Form feed (para documentos impresos)
`n	Nova liña
`r	Retorno de carro
`t	Tabulador horizontal
`v	Tabulador vertical (para documentos impresos)
`"	Comiñas simples
`""	Comiñas dobres. Unha alternativa e escribir catro comiñas dobres

Para escribir números hexadecimais, úsase a notación 0x<NÚMERO>. Por exemplo, 0x10 equivale ao número decimal 16. Tamén se pode converter un número hexadecimal en char para obter un carácter Unicode. Así, [char]0x0048 equivale a «H». Finalmente, todo isto pódese meter nunha cadea avaliando a expresión con \$(), como xa vimos. Desta maneira temos que "Rodolfo Ucha \$([char]0x00AE)" se interpreta como «Rodolfo Ucha ®».

Por outra banda, para escribir cantidades de bits, pódense usar as abreviaturas «KB», «MB», «GB», «TB», «PB» despois de un número. Por exemplo, temos 25MB, 89KB, 1TB, etc.

Unha característica especial de PowerShell son os chamados “*here-strings*” que permiten escribir expresións de varias liñas nunha instrución. Imaxinemos, que queremos escribir unha destas cadeas sen interpretar as variables. Podemos facelo nunha instrución usando o prototipo @'<EXPRESIÓN>', con comiñas simples. Por exemplo a expresión:

```
@'
```

```
$ (1+2)
```

```
$ (3+4)
```

```
$ (5+6)
```

```
'@
```

Devolve:

```
$ (1+2)
```

```
$ (3+4)
```

```
$ (5+6)
```

Pola contra, se queremos que as expresións se avalíen, empregaremos o prototipo @"<EXPRESIÓN>"@, con comiñas dobres. Por exemplo a expresión:

```
@"
```

```
$ (1+2)
```

```
$ (3+4)
```

```
$ (5+6)
```

```
"@
```

Devolve:

```
3
```

```
7
```

```
11
```

Por último, desde PowerShell 3.0, os símbolos `--%` serven para que PowerShell non avalíe caracteres especiais. Así:

```
Write-Host --% %USERNAME%,this=$algo{raro}
```

Devolve a cada:

```
«%USERNAME%,this=$algo{raro}»
```

Mentres que

```
Write-Host %USERNAME%,this=$algo{raro}
```

Devolve a cadea

```
«%USERNAME% this= raro»
```

Finalicemos entón cun pequeno script de proba:

```
Write-Host `
```

```
"Quero moitos `$ `n agora `t mesmo"
```

```
Write-Host "En xaponés $([char]0x4eca) "
```

```
$meuHexadecimal = 0xA4
```

```
Write-Host "O meu hexadecimal é: $($meuHexadecimal) "
```

```
$tamano = 25MB
```

```
Write-Host "O tamaño de memoria é $($tamano) "
```

```
Write-Host --% %USERNAME%,this=$algo{raro}
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
Quero moitos $
```

agora mesmo

En xaponés 

O meu hexadecimal é: 164

O tamaño da memoria é 26214400

--% %USERNAME% this= raro (en PowerShell 2.0)

%USERNAME%,this=\$algo{raro} (en PowerShell 3.0+)

PS C:\>

Tedes máis información sobre caracteres especiais en [1] e [2].

Referencias:

[1] Microsoft. "About Special Caracteres". En liña: <http://technet.microsoft.com/en-us/library/hh847835.aspx>. Accedido o 14-09-2019.

[2] Victor Zakharov. "PowerShell Special Characters and Tokens". En liña: <http://www.neolisk.com/techblog/powershell-specialcharactersandtokens>. Accedido o 14-09-2019.

1.22 Ler a entrada do usuario

Para ler a entrada do usuario, pódese usar o cmdlet Read-Host:

```
Read-Host [[-prompt] <OBJECTO>] [-asSecureString]
[<OUTROS_PARAMS>]
```

O prompt é o texto que se lle amosa ao usuario para pedirlle un valor. Co parámetro -asSecureString mostraranse asteriscos "*" cando o usuario escriba e se cifrará a cadea resultante, retornando un obxecto de tipo System.Security.SecureString, que como peculiaridade ten que non se pode imprimir (de intentalo simplemente escribirá "System.Security.SecureString").

É posible pasar dun SecureString a unha cadea cifrada e viceversa mediante os cmdlets ConvertFrom-SecureString e ConvertTo-SecureString. Incluso é posible pasar dunha cadea cifrada a unha cadea de texto plano mediante o seguinte código:

```
$CadeaDeBytes = [System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($ContrasinalCifrado)
$ContrasinalEnClaro = [System.Runtime.InteropServices.Marshal]::PtrToStringAuto($CadeaDeBytes)
```

Vexamos un exemplo:

```
$meulogin = Read-Host "Login"
```

```

$meupasswd = read-host "Contrasinal" -assecurestring

Write-Host "Meu passwd seguro:" $meupasswd

$passwd2 = ConvertFrom-SecureString $meupasswd

Write-Host "Meu passwd cifrado:" $passwd2

$passwd3 = ConvertTo-SecureString $passwd2

Write-Host "Contrasinal volto a asegurar:" $passwd3

$CadeaDeBytes =
[System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($meuPasswd)

$ContrasinalEnClaro =
[System.Runtime.InteropServices.Marshal]::PtrToStringAuto($CadeaDeBytes)

Write-Host "Contrasinal en claro:" $ContrasinalEnClaro

```

A execución debería devolver un resultado coma o seguinte:

```

PS C:\> .\script.ps1

Login: rafaellg

Contrasinal: abc123.

Meu passwd seguro: System.Security.SecureString

Meu                passwd                cifrado:
01000000d08c9ddf0115d1118c7a00c04fc297eb01000000a67366f53299cb4b
9110142f51fb186b0000000002000000000106600000001000020000000c172
6d999db89a2af43ee086faa097f0cca7b489821faabb82edd5768915631d0000
00000e800000000200002000000004cea06b2d08cded096c2df97737323ff549f
fdf4483e7b129a7db1966d40d37810000000b100f0697bee2c7aabee3f6bc853
d23c4000000019182c1d3355774b6d4038cb2c77ec01c19dc5eabd4c66b5e3c4
25f43103b290e472a7373439de5acf9bd4dc06226ece5631909d3741f377b3fb
472c6d0917ed

Contrasinal volto a asegurar: System.Security.SecureString

Contrasinal en claro: abc123.

PS C:\>

```

Aínda así, para pedir un usuario e un contrasinal temos outro mecanismo que é o cmdlet Get-Credential. Este mecanismo amosa unha ventá que pide o par usuario/contrasinal e devolve un obxecto de tipo PSCredential. Este obxecto ten, entre outros, un atributo UserName que representa o usuario e un atributo Password que representa o contrasinal seguro. Tamén

dispón dun método `getNetworkCredential()` que nos permite acceder ao password en claro, obtendo o atributo `Password` do obxecto devolto por este método. Vexamos un exemplo:

```
$credenciais = Get-Credential -Message "Introduza o seu usuario e contrasinal"

Write-Host "Usuario: " $credenciais.UserName

Write-Host "Contrasinal seguro:" $credenciais.Password

Write-Host "Contrasinal en claro:" $credenciais.getNetworkCredential().Password
```

E a execución devolvería o seguinte resultado:

```
PS C:\> .\script.ps1

Introducimos usuario rafaellg e contrasinal abc123.

Usuario: rafaellg

Contrasinal seguro: System.Security.SecureString

Contrasinal en claro: abc123.

PS C:\>
```

Podedes ver un tutorial sobre estes mecanismos en [1].

Referencias:

[1] Microsoft. "Working with Passwords, Secure Strings and Credentials in Windows PowerShell". En <https://social.technet.microsoft.com/wiki/contents/articles/4546.working-with-passwords-secure-strings-and-credentials-in-windows-powershell.aspx>. Accedido o 29-10-2019.

1.23 Máis sobre redireccións

Aínda que por defecto todas as mensaxes se amosan por pantalla, PowerShell representa as canles de saída mediante números, ao estilo de bash. Por defecto, temos que "1" representa a saída estándar (éxito), mentres que "2" representa os erros. Mais desde PowerShell 3.0, incorporáronse outros descritores como "3", que representa os avisos (warnings), "4", que representa a saída verbosa e "5", que representa as mensaxes de depuración (debug). Por conveniencia, "*" representa todos os descritores do 1 ao 5.

Como cabe esperar, é posible redirixir a saída dos comandos a estas canles mediante redireccións. Así `M>NF` redirixe a canle M ao ficheiro NF borrando o contido que tivera previamente. Por exemplo, `1>saída.txt` redirixe a saída estándar ao ficheiro saída.txt e borra o seu contido previo se o houbese. Por outra parte, `M>>NF` redirixe a canle M ao ficheiro NF en modo engadir ao final (append), sen borrar o contido existente, e `M>&N` redirixe a canle M á canle N. Por exemplo `2>&1` redirixe os erros á saída estándar. Porén, en PowerShell 2.0, a cadea `1>&2` devolve un erro.

Ademais, temos cmdlets como *Out-File*, *Tee-Object*, *Start-Transcript*, etc., que presentan funcionalidades similares á redirección pero con opcións.

Como xa vimos, tamén, para pasar a saída dun comando á entrada doutro, pódese usar o carácter «|». Por exemplo:

```
Get-Process | Where-Object {$_.PriorityClass -eq "Normal"}
```

Por último, para ler un ficheiro, non queda outra que usar os cmdlets *Get-Content* e similares, xa que, a diferenza de bash, PowerShell non ten o descritor 0 para a chamada «entrada estándar», nin permite redirixir a entrada desde un ficheiro con «<» (actualmente este operador reservado para uso futuro).

Vexamos un exemplo de redirección:

```
Get-ChildItem C:\windows >> "meu_log.txt"

Start-Transcript -path "meu_log2.txt"

Write-Host "Ola"

Stop-Transcript

Get-Content -path "meu_log2.txt"

Write-Error "Hola" > "error_ola.txt"

Write-Error "Hola" 2>&1 > "error_ola2.txt"
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1

La transcripción ha comenzado. El archivo de salida es
meu_log2.txt

Ola

La transcripción se ha detenido. El archivo de salida es C:\
meu_log2.txt

*****

... (Aquí viría toda a transcripción de PowerShell)

*****

C:\script.ps1 : Ola

En línea: 1 Carácter: 13

+ .\script.ps1 <<<<

+ ...
```

PS C:\>

Podedes ver máis información sobre redireccións en [1][2] e [3].

Referencias:

[1] Microsoft. "About Redirection". En liña: <https://technet.microsoft.com/es-es/library/hh847746.aspx>. Accedido o 14-09-2019.

[2] SS64.com. "Redirection". En liña: <https://ss64.com/ps/syntax-redirection.html>. Accedido o 14-09-2019.

[3] Microsoft. "Get-Content". En liña: <https://docs.microsoft.com/en-gb/powershell/module/Microsoft.PowerShell.Management/Get-Content>. Accedido o 14-09-2019.

1.24 O condicional "if"

O condicional "if" serve para elixir que facer se se da unha certa condición. Como en moitas linguaxes de programación imperativa, a súa estrutura é a seguinte [1]:

```
if (CONDICIÓN) {CONSECUENTE}

[ elseif (CONDICIÓN) {CONSECUENTE} ]

[ else {ALTERNATIVA} ]
```

Se algunha condición se avalía como verdadeira, o seu consecuente se executa e finaliza o bloque. Se non, a alternativa se executa se existe.

A condición pode levar calquera dos operadores de comparación de PowerShell, e que xa fomos vendo nas seccións dedicadas a operacións aritméticas, cadeas de texto, arrays, expresións regulares, etc. [2].

Un exemplo típico de script é facer a comprobación dos parámetros que se lle pasan. Vexamos como facelo.

```
if ($args.Count -eq 0) {

    Write-Host "Non hai parámetros"

} elseif ($args.Count -eq 1) {

    Write-Host "Só hai un parámetro"

} else {

    Write-Host "Hai varios parámetros"

}
```

A execución dá o seguinte resultado:

PS C:\> .\script.ps1

Non hai parámetros

```
PS C:\> .\script.ps1 ola
```

Só hai un parámetro

```
PS C:\> .\script.ps1 ola adeus
```

Hai varios parámetros

```
PS C:\>
```

Referencias:

[1] SS64.com “If”. En liña: <https://ss64.com/ps/if.html>. Accedido o 16-09-2019.

[2] SS64.com “Comparison operators”. En liña: <https://ss64.com/ps/syntax-compare.html>.
Accedido o 16-09-2019.

1.25 O condicional “switch”

Podemos considerar o condicional “switch” como azucre sintáctico para cando hai moitas alternativas nun “if”. A súa estrutura é [1]:

```
switch [ -regex | -wildcard | -exact ] [-CaseSensitive] ($item)
{
    valor_1 { INSTRUCCIÓNS }
    [valor_2 { INSTRUCCIÓNS }]
    ...
    [valor_n { INSTRUCCIÓNS }]
    [ default { INSTRUCCIÓNS } ]
}
```

Neste tipo de construción, execútanse os comandos de todos os valores que verifiquen o valor da variable «\$item». Respecto a isto, hai que ter coidado porque, por defecto, a comparación de «\$item» cos casos non é sensible a maiúsculas. A opción -CaseSensitive faina sensible. Ademais, por defecto a comparación ten que ser exacta. Só a opción -wildcard permite o uso de comodíns. Finalmente, a opción -regex permite o uso de expresións regulares.

Ao igual que en linguaxes como C e Java, se un caso remata coa orde «break» e se executa, sae do «switch» e os demais casos que verifiquen o valor de «item» xa non se executan. E se se inclúe un caso «default», este execútase se ningún outro verificou a expresión pasada.

Vexamos un exemplo:

```
switch -wildcard -casesensitive ($args[0]) {
    'o*' {Write-Host "Comeza por o"}
```

```

'a*' {Write-Host "Comeza por a"}

'o' {Write-Host "Remata en o"}

'a' {Write-Host "Remata en a"}

'???' {Write-Host "Ten 3 caracteres"; break;}

default {Write-Host "Ningunha das anteriores"}

}

```

A execución dá o seguinte resultado:

```

PS C:\> .\script.ps1 ola

Comeza por o

Remata en a

Ten 3 caracteres

PS C:\> .\script.ps1 OLA

Ten 3 caracteres

PS C:\> .\script.ps1 adeus

Comeza por a

PS C:\> .\script.ps1 u

Ningunha das anteriores

```

Como exercicio, podedes probar que pasa se quitades as opcións -casesensitive e -wildcard.

Referencias:

[1] SS64.com “Switch”. En liña: <https://ss64.com/ps/switch.html>. Accedido o 16-09-2019.

1.26 Os bucles “while”, “do ... while” e “do ... until”

O bucle “while” avalía a condición e, se esta é certa, executa as ordes que hai no seu corpo unha e outra vez ata que a condición deixa de verificarse. A súa estrutura é [1]:

```

while (CONDICIÓN) {

    COMANDOS

}

```

Pola súa parte, o bucle “do ... while” é similar ao bucle while coa única diferenza de que as ordes no seu corpo se executan polo menos unha vez e a condición se comproba despois. Polo tanto, temos que do...while = COMANDOS + while. A súa estrutura é [2].

```

do {

```

COMANDOS

```
} while (CONDICIÓN)
```

Finalmente, o bucle “do ... until” é similar o bucle “do...while”, coa única diferenza de que só se executan as ordes do corpo ata que a condición se verifica. Polo tanto, temos que do ... until(CONDICIÓN) = do...while(!CONDICIÓN). A súa estrutura é [2]:

```
do {  
    COMANDOS  
} until (CONDICIÓN)
```

Vexamos uns exemplos destes bucles. Como podedes comprobar, os tres son equivalentes:

Exemplo 1:

```
$i=1  
while ($i -le 3) {  
    Write-Host $i  
    $i++  
}
```

Exemplo 2:

```
$i=1  
do {  
    Write-Host $i  
    $i++  
} while ($i -le 3)
```

Exemplo 3:

```
$i=1  
do {  
    Write-Host $i  
    $i++  
} until ($i -gt 3)
```

A execución de calquera dos bucles dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

1

2

3

PS C:\>

Como exercicio, podedes substituír o «3» por «0» na condición de cada bucle e ver como o primeiro non se executa, mentres que os outros dous execútanse polo menos unha vez.

Referencias:

[1] SS64.com. "While". En liña: <https://ss64.com/ps/while.html>. Accedido o 16-09-2019.

[2] SS64.com. "Do". En liña: <https://ss64.com/ps/do.html>. Accedido o 16-09-2019.

1.27 O bucle "for"

O bucle for execútase mentres unha variable numérica satisfaga unha condición. É preciso especificar os seguintes elementos:

- Con que valor comeza a variable.
- A condición para que a seguinte iteración se execute.
- A forma na que a variable se incrementa en cada iteración.

A estrutura é a seguinte [1]:

```
for (INICIO;CONDICIÓN;INCREMENTO) {  
    COMANDOS  
}
```

Vexamos agora un par de scripts de exemplo:

```
for ($i=0; $i -le 10; $i++) {  
    "10 * $i = " + (10 * $i)  
}
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
10 * 1 = 10
```

```
10 * 2 = 20
```

```
...
```

```
10 * 10 = 100
```

```
PS C:\>
```

Vexamos outro exemplo con outro tipo de lista:

```
$col = @("Vermello", "Laranxa", "Verde", "Amarelo", "Azul")  
for ($i=0;$i -lt $col.Length;$i++) {  
    $col[$i]  
}
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1  
  
Vermello  
  
Laranxa  
  
Verde  
  
Amarelo  
  
Azul  
  
PS C:\>
```

Referencias:

[1] SS64.com. "For". En liña: <https://ss64.com/ps/for.html>. Accedido o 16-09-2019.

1.28 O bucle "ForEach-Object"

O bucle ForEach-Object itera sobre unha colección predefinida de valores. A diferenza do bucle "for", non se necesita incrementar manualmente ningún contador. O obxecto procesado en cada iteración se almacena automaticamente na variable especial \$_. A estrutura sintáctica do bucle é a seguinte [1]:

```
ForEach-Object -InputObject COLECCIÓN {  
    COMANDOS  
}
```

Alternativamente, a colección pódese pasar por pipeline:

```
COLECCIÓN | ForEach-Object {  
    COMANDOS  
}
```

Desde PowerShell 3.0 en lugar do bloque de script, pódese escribir unha «declaración de operación». Isto é, escríbese simplemente un atributo ou método en lugar de comandos. Por exemplo, en lugar de:

```
Get-Process | ForEach-Object {$_.ProcessName}
```

Pódese escribir:

```
Get-Process | ForEach-Object ProcessName
```

Por certo, `ForEach-Object` ten un alias chamado «%». Tamén hai outra sentenza similar chamada `ForEach` [2], mais esta último non é exactamente a mesma sentenza xa que non pode pasar a súa saída ao pipeline, senón que é preciso gardala en variables. Ademais, se `ForEach` recibe a entrada do pipeline, carga todos os elementos en memoria antes de procesalos. Isto é máis rápido pero máis custoso en memoria [3].

Como práctica, imos ver catro bucles `ForEach-Object` practicamente alternativos:

1. `Get-Process | ForEach-Object {$_.ProcessName}`
2. `Get-Process | ForEach {$_.ProcessName}`
3. `Get-Process | % {$_.ProcessName}`
4. `Get-Process | % ProcessName`

A execución de calquera dos bucles devolverá un resultado coma o seguinte:

```
PS C:\> .\script.ps1

conhost

conhost

csrss

...

WmiPrvSE
```

O primeiro e o terceiro son equivalentes, mentres que os outros teñen lixeiras diferenzas. O segundo, por exemplo, vaise executar máis rápido, xa que usa `ForEach`. O cuarto, aínda que é similar ao primeiro e o terceiro, só se pode executar en PowerShell 3.0 ou superior.

Por certo, un *ForEach-Object* pode ter 3 parámetros chamados “begin”, “process” e “end”.

1. O parámetro “begin” execútase unha soa vez ao comezo do proceso. Os obxectos da colección non están dispoñibles nese punto.
2. O parámetro “process” execútase unha vez por cada parámetro pasado no pipeline. O parámetro da iteración actual está accesible mediante a variable especial `$_`.
3. O parámetro “end” se executa unha soa vez ao final del proceso.

Un `ForEach-Object` sen “begin”, “process” e “end” se considera igual que un que só ten “process”. `ForEach`, en cambio, non pode ter ningún destes bloques. Para rematar, imos ver un exemplo disto no que procesamos os eventos do log, os cales se poden obter co cmdlet `Get-EventLog`. A idea deste script é comezar indicando a data (cmdlet `Get-Date`) na que se fai o procesamento, facelo, e volver a imprimirla data para ver canto tardou.

```
$Events = Get-EventLog -LogName System -Newest 1000

$Events | ForEach-Object `
```



```
-Begin {Write-Host $(Get-Date)} `
-Process {Out-File -Filepath e.txt -Append -InputObject
$_Message} `
-End {Write-Host $(Get-Date)}
```

A execución dá o seguinte resultado e ademais crea un fichero chamado e.txt que conterá os eventos:

```
PS C:\> .\script.ps1
```

```
05/02/2018 16:06:16
```

```
05/02/2018 16:06:17
```

Máis sobre ForEach-Object na web de Microsoft [4].

Referencias:

[1] SS64.com. “ForEach-Object”. En liña: <https://ss64.com/ps/foreach-object.html>. Accedido o 16-09-2019.

[2] SS64.com. “ForEach”. En liña: <https://ss64.com/ps/foreach.html>. Accedido o 16-09-2019.

[3] Dr. Scripto. “Getting to know ForEach and ForEach-Object”. En liña: <https://devblogs.microsoft.com/scripting/getting-to-know-foreach-and-foreach-object/>. Accedido o 16-09-2019.

[4] Microsoft. “ForEach-Object”. En liña: <https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/foreach-object>. Accedido o 16-09-2019.

1.29 Os comandos “break” e “continue”

Para rematar a parte de sentenzas iterativas, veremos os comandos break e continue sen pararnos moito, xa que son iguais que no resto de linguaxes de programación. O comando “break” serve para abandonar un bucle prematuramente sen que se dera a condición de terminación. Pola súa parte, o comando “continue” serve para forzar a seguinte iteración do bucle. Así, lese o seguinte valor e fanse as comprobacións necesarias. Vexamos un par de exemplos:

```
for ($i = 1; $i -le 10; $i++) {
    if ($i -eq 5) {
        Write-Host "5 atopado"
        break
    }
    Write-Host "Non é útil:" $i
```

```
}
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
Non é útil: 1
```

```
Non é útil: 2
```

```
Non é útil: 3
```

```
Non é útil: 4
```

```
5 atopado
```

```
PS C:\>
```

Vexamos agora o exemplo de continue:

```
for ($i = 1; $i -le 10; $i++) {  
    if ($i -eq 5) {  
        Write-Host "Saltamos o 5"  
        continue  
    }  
    Write-Host "Procesando:" $i  
}
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
Procesando: 1
```

```
Procesando: 2
```

```
Procesando: 3
```

```
Procesando: 4
```

```
Saltamos o 5
```

```
Procesando: 6
```

```
Procesando: 7
```

```
Procesando: 8
```

```
Procesando: 9
```

Procesando: 10

1.30 Funcións e filtros

Unha función é unha secuencia de comandos que se pode executar en calquera punto do script. Basta con invocar o seu nome como se dun comando se tratase. Para declarar unha función pódese empregar a seguinte sintaxe [1]:

```
function [ÁMBITO:]FUNCIÓN [(PARÁMETROS)] {COMANDOS}
```

Como se pode ver arriba, as funcións poden ter parámetros, e de feito PowerShell ofrece varios mecanismos para traballar con eles [2]. O primeiro é usar parámetros posicionais, os cales non teñen nome, de forma que accederemos a eles empregando o array especial `$args` que xa coñecemos. Mais é preciso ter en conta que os valores de `$args` non son os mesmos que se lle pasan ao noso script na mesma variable, senón que serán os que se lle pasen á función. Un exemplo de acceso é o seguinte:

```
function Obter-NomeConExtension {  
    $nome = $args[0] + ".txt"  
    ...  
}
```

Como podedes ver, non hai un parámetro explicitamente declarado, mais pódese acceder a el.

Tamén se poden empregar parámetros con nome, poñéndoo entre parénteses entre o nome da función e a chave de apertura do corpo, separados por comas. Incluso se lles pode dar un valor por defecto. Por exemplo:

```
function dicir ([string]$frase, [int]$veces=1) {COMANDOS}
```

Unha alternativa completamente equivalente á forma anterior (e preferida por Microsoft [1]) sería usando o valor especial *params* que xa vimos. Neste caso, tamén aplica a regra de que o acceso a *params* ten que ser a primeira liña do corpo da función. Os parámetros poden ter modificadores para indicar se son obrigatorios ou opcionais, se se poden tomar do pipeline, etc.

```
function loguear {  
    Param(  
        [Parameter(Mandatory=$true)][String] $Usuario,  
        [Parameter(Mandatory=$false)][String]$Contrasinal  
    )  
    ...  
}
```

Ao chamar á función os parámetros se teñen que pasar separados por espazos, non entre parénteses nin separados por comas.

```
loguear -Usuario "Rafa" -Password "abc123."
```

Finalmente, desde PowerShell 3.0, é posible usar o *parameter splatting*, que xa vimos, para pasar parámetros a unha función. Así, se por exemplo, se queremos chamar ao cmdlet *Get-Command* e temos todos os parámetros nunha táboa hash chamada *@Args*, podemos facer:

```
Get-Command @Args
```

E así a *Get-Command* xa se lle pasarían todos os parámetros contidos en *@Args*.

Por outra banda, unha función só existe no ámbito no que se definiu. Por defecto, estas non se ven fóra do script (p.ex.: na liña de comandos), mais é posible acadar isto definindo a función como de ámbito global. Para declarar unha función con este ámbito, non temos máis que engadir a cadea "global:" diante do nome da función. Por exemplo:

```
function global:dicir ([string]$frase, [int]$veces=1) {COMANDOS}
```

Existen bastantes máis modificadores de ámbito para as funcións, mais quedan fóra do alcance deste tutorial. Para obter máis detalles sobre este tema podedes visitar a web de Microsoft [3].

Outra cousa que podemos querer facer é imprimir o contido dunha función. Para isto non temos que invocar algún destes comandos:

```
Get-Content function:NOMBRE_FUNCIÓN  
  
${function:NOMBRE_FUNCIÓN}  
  
(Get-Command NOMBRE_FUNCIÓN).definition
```

Vexamos un script de exemplo:

```
function decir {  
    Write-Host "Dixo: $($args[0])"  
}  
  
function berrar ([string]$frase) {  
    Write-Host "Berrou: $($frase.ToUpper())"  
}  
  
dicir ola  
  
berrar adeus  
  
cat function:dicir
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
Dixo: ola
```

```
Berrou: ADEUS
```

```
Write-Host "Dixo: $($args[0])"
```

```
PS C:\>
```

Outra característica das funcións é que poden procesar obxectos pasados polo pipeline. Ao igual que o bucle `ForEach-Object`, unha función pode ter 3 bloques especiais llamados “begin”, “process” e “end”:

- O bloque “begin” execútase unha soa vez e cando o fai, os parámetros pasados polo pipeline aínda non se leron.
- O bloque “process” execútase unha vez por cada parámetro pasado no pipeline. O parámetro da iteración actual está accesible mediante a variable especial `$_`.
- O bloque “end” execútase unha soa vez ao final do proceso.

Unha función sen “begin”, “process” e “end” se considera igual que aquela que só ten “end”.

Vexamos un exemplo de script con estes bloques:

```
function dicir {  
    begin {  
        Write-Host "E dixo: "  
    }  
    process {  
        Write-Host $_  
    }  
    end {  
        Write-Host "Fin da cita"  
    }  
}  
  
"Ola","Adeus","Grazas" | dicir
```

O resultado da execución sería:

```
PS C:\> .\script.ps1
```

```
E dixo:
```

```
Ola
```

Adeus

Grazas

Fin da cita

Un último aspecto a considerar das funcións é que teñen una variable especial chamada `$input` que almacena a colección de valores que se lle pasou polo pipeline. Recordemos que en ningún caso estes valores están dispoñibles durante o bloque “begin”. Mais, se non houbera un bloque “process”, a colección completa está dispoñible no bloque “end” na variable `$input`. Se pola contra houbera un bloque “process”, os valores vanse quitando de `$input` a medida que se van procesando, co que `$input` estará baleiro no “end”.

En ocasións poderedes ver que ademais de funcións, en PowerShell hai “filtros” (filters). Un filtro é como unha función que só ten o bloque process. Para declarar un filtro pódese utilizar a seguinte sintaxe:

```
filter [ÁMBITO:]FILTRO {COMANDOS}
```

Así, un exemplo parecido ao do script anterior, pero sin “begin” nin “end” sería:

```
filter dicir {  
    Write-Host $_  
}
```

Referencias:

[1] Microsoft. “About functions”. En liña: https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_functions. Accedido o 17-09-2019.

[2] Microsoft. “About functions advanced parameters”. En liña: https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_functions_advanced_parameters. Accedido o 17-09-2019.

[3] Microsoft. “About Scopes” https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_scopes. Accedido o 17-09-2019.

1.31 Erros

En PowerShell, podemos obter erros por diversos motivos, pero en xeral estes se dividen en dous tipos. O primeiro é o dos erros críticos, que son aqueles que fan que o script non se siga executando. Un exemplo sería facer unha chamada a un cmdlet que non existe. Outro sería un erro de sintaxe. O outro grupo é o dos erros non críticos, que son aqueles que permiten que o script se siga executando. Exemplos deste tipo serían que non se atope un ficheiro ou que falten permisos. É posible que queiramos manexar os erros non críticos para continuar a execución do script.

En PowerShell hai unha variable especial chamada `$error` de tipo `ArrayList` de `ErrorRecord` que almacena os últimos error ocorridos. Este array está baleiro ao arrancar a sesión. Despois, vaise enchendo a medida que ocorren erros, de forma que o máis recente se almacena na posición 0 e así sucesivamente. Os obxectos de tipo `ErrorRecord` teñen un atributo chamado `InvocationInfo` que contén o contexto no que o comando deu o erro, se é que este está dispoñible. Á súa vez, os obxectos de tipo `ErrorRecord` teñen un atributo chamado `Exception` que contén o erro real, tal e como se rexistrou. Hai varios tipos de `Exception`, e ver o tipo de cada una é moi ilustrativo para resolver o problema que teñamos. Por exemplo, se atopamos:

```
System.Management.Automation.CommandNotFoundException
```

Podemos ter claro que o problema é que o cmdlet en cuestión non existe.

Unha `Exception`, ademais ten un atributo chamado `StackTrace` que contén máis información sobre a función que chamou ao comando que deu erro. A maiores, unha `Exception` tamén pode conter outra `Exception`, que se pode ver no membro `InnerException`.

Vexamos toda esta estrutura a través dun script, aínda que mellor recomendaría executar as sentenzas unha a unha e ir analizando o que van devolvendo, xa que a súa saída é demasiado longa para que a poñamos aquí.

```
Get-ChildItem "G:\NonExiste"
```

```
$error.Count
```

```
$error[0]
```

```
$error[0] | Get-Member
```

```
$error[0].Exception
```

```
$error[0].Exception | Get-Member
```

```
$error[0].Exception.StackTrace
```

1.32 A variable `$ErrorActionPreference`

PowerShell ten unha variable automática chamada `$ErrorActionPreference` que indica que facer en caso de que ocorra un erro non crítico durante a execución das sentenzas. As accións dispoñibles son:

- `SilentlyContinue`: a execución continúa e as mensaxes de erro suprimense.
- `Stop`: para a execución como se fose un error crítico.
- `Continue`: amosa o erro e continúaase a execución. É a opción por defecto.
- `Inquire`: pregunta ao usuario que facer.
- `Ignore`: ignórase o erro e nin sequera é logueado na saída de erro. Esta opción está dispoñible a partir de PowerShell 3.0+

Se queremos cambiar esta preferencia para un so cmdlet, tamén podemos facelo mediante o parámetro `-ErrorAction` (alias `"-ea"`). Vexamos un exemplo de script.

```
#NonExiste é un directorio que non existe
```

```

#G: é unha unidade que non existe

#Con inquire preguntamos ao usuario que facer

Get-ChildItem "G:\NonExiste" -ErrorAction "Inquire"

#A partir de agora, todos os erros serán tratados como críticos.

$ErrorActionPreference = "Stop"

#Pero o seguinte non porque ten -ErrorAction "Continue"

Get-ChildItem "G:\NonExiste" -ErrorAction "Continue"

#E agora un erro ascendido a crítico polo Stop de arriba

Get-ChildItem "G:\NonExiste"

Write-Host "Fin do script"

```

Nota: Lembra de volver a poñer `$ErrorActionPreference = "Continue"` ao finalizar ou de reiniciar a sesión de PowerShell tras empregar este script. Se non, as seguintes prácticas darán erros críticos cando non deberían.

1.33 Manexo de excepcións

Para manexar erros críticos (ou non críticos se teñen o seu `-ErrorAction` establecido a “Stop”), podemos facer uso dos bloques `try...catch...finally...` ao máis puro estilo de linguaxes orientadas a obxectos como Java.

```

try {

COMANDOS

}

catch [EXCEPCIÓN] {

COMANDOS A EXECUTAR EN CASO DE ERRO

}

finally {

COMANDOS QUE SEMPRE SE EXECUTAN

}

```

Como é costume nestas linguaxes, o bloque “try” é como una secuencia de comandos normal, pero se se producise un erro durante a súa execución, se abandonaría e se pasaría a executar o bloque “catch” correspondente á excepción ocorrida, se o houbese. Non pode haber bloque “try” sen polo menos un bloque “catch” ou o bloque “finally”.

En canto ao bloque “catch”, pode haber varios deles, cada un indicando que facer en caso de que ocorra o erro especificado entre corchetes na súa cabeceira. De non especificar un tipo de erro, calquera erro que ocorra no “try” pasará por ese bloque “catch”.

Despois, pasaríase a executar o bloque “finally” se o houbese. O bloque finally execútase sempre, independentemente de si se produciu erro ou non (de feito, execútase incluso se o usuario pulsou CTRL+C). Este bloque serve, polo xeral, para liberar recursos previamente reservados (p. ex.: una conexión a unha base de datos, etc.).

Vexamos un script de exemplo:

```
try {  
    Write-Host "Procesando ficheiro"  
  
    #Código perigoso aquí  
  
    $content = Get-Content -Path "C:\Nonexiste.txt" -ErrorAction  
Stop  
  
    Write-Host "Ficheiro procesado"  
} catch {  
    # Aquí manéxase o erro  
  
    Write-Host "O ficheiro non existe!!!" -ForegroundColor Red  
} finally {  
    # Código que sempre se executa  
  
    Write-Host "Código de finally"  
}
```

Como cabería esperar, a execución dá o seguinte resultado:

```
PS C:\> .\script.ps1  
  
Procesando ficheiro  
  
O ficheiro non existe!!!  
  
Código de finally  
  
PS C:\>
```

Nota: vemos que a liña que imprime «Ficheiro procesado» non se executa. Cando ocorre un erro vaise directamente ao bloque “catch” correspondente e logo ao “finally” se o houbese.

Como xa se dixo, os bloques “catch” manexan as posibles excepcións. Por exemplo:

```
} catch [System.io.FileNotFoundException] {
```

Se se quere manexar varias excepcións da mesma forma, pódense poñer nunha lista separada por comas tralo «catch». Así teríamos:

```
} catch [EXCEPCIÓN_1], ..., [EXCEPCIÓN_N] {
```

E como xa dixemos, pódese ter varios “catch” un tras outro, para manexar diversos error de diferente maneira:

```
} catch [EXCEPCIÓN_1] {  
    MANEXADOR_1  
  
} catch [EXCEPCIÓN_2] {  
    MANEXADOR_2  
  
}
```

E, como xa se dixo, ao final podemos ter un “catch” xenérico para calquera erro.

De termos varios bloques “catch”, non se pode ter un máis xeral diante dun máis particular. Por exemplo, facer isto sería ilegal.

```
} catch {  
    Write-Host "Catch 1"  
  
} catch [System.io.FileNotFoundException] {  
    Write-Host "Catch 2"  
  
}
```

Por certo, nun bloque “catch”, a variable automática \$_ contén o erro. Podedes probar a imprimir algúns dos seus membros como fixemos anteriormente.

Unha cousa mais sobre excepcións. Supoñendo que un erro sexa non crítico e se eleve a crítico mediante o parámetro “-ErrorAction Stop”, a excepción que hai que atrapar non é a orixinal dese comando, senón que se substitúe por outra excepción chamada:

```
System.Management.Automation.ActionPreferenceStopException
```

Pero neste caso a “maxia negra” de PowerShell fará que \$_ conteña o erro coa excepción orixinal [1].

Vexamos outro script de exemplo:

```
try {  
    Write-Host "Procesando ficheiro"  
  
    $content = Get-Content -Path "C:\Nonexiste.txt" -ErrorAction  
    "Stop"
```

```

Write-Host "Ficheiro procesado"
}
[System.Management.Automation.ActionPreferenceStopException] {
    $_.Exception.GetType().FullName
    Write-Host "Parado polo stop!!!" -ForegroundColor Red
} catch [System.Management.Automation.ItemNotFoundException] {
    $_.Exception.GetType().FullName
    Write-Host "O ficheiro non existe!!!" -ForegroundColor Red
} catch [System.Management.Automation.RuntimeException] {
    Write-Host "Ocorreu un erro grave" -ForegroundColor Red
}

```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

Procesando ficheiro

System.Management.Automation.ItemNotFoundException

Parado polo stop!!!

```
PS C:\>
```

Vemos como o sistema atrapou unha *ActionPreferenceStopException* mais á hora de imprimila temos que é unha *ItemNotFoundException*.

Un problema que non consideramos ata o de agora é que no exemplo anterior sabiamos qué excepcións queríamos manexar, mais os cmdlets non adoitan documentar que excepcións poden producir. Así, non queda outra que obter o nome da excepción chamando a `$error[0].Exception.GetType().FullName` tras ocorrer o erro, para que este aínda estea gardado en `$error[0]`, ou ben facer o mesmo sobre a variable `$_` nun bloque “catch” xenérico. Así, se non hai documentación, non queda máis remedio que forzar posibles situacións de erro nun script de proba. Hai listas de excepcións de .NET en Internet [2][3].

O que nunca se debe facer é «tragarse» excepcións, é dicir, recollelas cun “catch” e non facer nada, nin sequera log, xa que isto fai o código moito máis difícil de arranxar se houbese problemas no futuro.

Para finalizar, unha limitación dos bloques `try...catch...finally...` é que non poden atrapar erros en procesos externos. En caso de que queiramos facer isto, teremos que empregar a variable especial `$LastExitCode`, que garda o código de saída do último comando [4]. Nesta variable, un valor 0 adoita indicar éxito e cada valor maior que 0 simbolizará un erro distinto.

Porén, isto non é sempre así, polo que é preciso ler a documentación de cada comando. Por exemplo, no comando “robocopy” o valor 1 tamén indica unha saída sen erros. Probemos a variable \$LastExitCode con este comando:

```
robocopy.exe "C:\NoExiste" "C:\NewDestination" "*.*" /R:0  
Write-Host $LastExitCode
```

Referencias:

[1] Varios usuarios de StackOverflow.com. “PowerShell try/catch/finally”. En liña: <https://stackoverflow.com/questions/6779186/>. Accedido o 19-09-2019.

[2] Kevin Marquette “All .NET Exception List.”. En liña: <https://kevinmarquette.github.io/2017-04-07-all-dotnet-exception-list/>. Accedido o 19-09-2019.

[3] Mike Vallotton. “.NET Exceptions (all of them)”. En liña: <https://mikevallotton.wordpress.com/2009/07/08/net-exceptions-all-of-them/>. Accedido o 19-09-2019.

[4] Keith Babinec. “An Introduction to Error Handling in PowerShell”. En liña: <https://blogs.msdn.microsoft.com/kebab/2013/06/09/an-introduction-to-error-handling-in-powershell/>. Accedido o 19-09-2019.

1.34 O cmdlet “throw”

En ocasións, pode que non queiramos manexar un erro crítico dentro dunha función. Neste caso, o comando throw, «lanza» o erro para que o manexe o script que chamou á función onde se produciu o erro [1]. A sintaxe é:

```
throw <MENSAXE>
```

E a excepción lanzada é de tipo *RuntimeException*.

Tamén se pode indicar o tipo de excepción que se desexa lanzar poñéndoa entre o throw e a mensaxe. Por exemplo:

```
throw [System.IO.FileNotFoundException] "No se atopou: $path"
```

Neste caso é obrigatorio incluír unha mensaxe.

Tamén é posible crear excepcións construíndo un obxecto da clase á que pertence a excepción. Unha maneira de facer isto é mediante o cmdlet *New-Object*:

```
throw (New-Object -TypeName "System.IO.FileNotFoundException")
```

E, se se quere, pódese engadir unha mensaxe co parámetro -ArgumentList.

Outra maneira de facelo é mediante o construtor new(), ao cal podemos acceder mediante o operador de chamada a membro estático, mais esta solución só está dispoñible a partir de PowerShell 5.0:

```
throw [System.IO.FileNotFoundException]::new()
```

Neste caso, pódese incluír unha mensaxe dentro dos parénteses de new().

A diferenza destes dous mecanismos que vimos anteriormente é que con estes a mensaxe é opcional.

Vexamos un script de exemplo:

```
function FacerAlgo {  
    throw "Ocorreu un erro"  
}  
  
try {  
    Write-Host "Imos facer algo"  
    FacerAlgo  
} catch {  
    $_.Exception.GetType().FullName  
}
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1  
  
Vamos a hacer algo  
  
System.Management.Automation.RuntimeException  
  
PS C:\>
```

Como exercicio, probade a engadir un tipo á excepción ao lanzala.

Referencias:

[1]. Kevin Marquette. "PowerShell Exceptions: Everything You Ever Wanted to Know". En liña: <https://kevinmarquette.github.io/2017-04-10-Powershell-exceptions-everything-you-ever-wanted-to-know/>. Accedido o 19-09-2019.

1.35 O cmdlet "Write-Error"

O cmdlet *Write-Error* lanza error non críticos [1]:

```
Write-Error "<MENSAXE>"
```

Ademais, mediante o parámetro -Exception pódese elixir o tipo de excepción que se quere lanzar.

```
Write-Error -Exception ([EXCEPCIÓN]"<MENSAXE>")
```

```
Write-Error -Exception (New-Object -TypeName "<EXCEPCIÓN>") -  
Message "<MENSAXE>"
```

```
Write-Error -Exception ([EXCEPCIÓN]::new())
```

No último caso, que só está dispoñible a partir de PowerShell 5.0, a mensaxe pode ir no parámetro -Message ou dentro de new().

Tamén é posible incluír un erro en lugar dunha excepción con -ErrorRecord en lugar de -Exception. Ademais, pódese converter o erro en crítico engadindo o parámetro -ErrorAction "Stop". Porén, debería usarse o cmdlet throw en lugar de facer isto.

Vexamos un script de exemplo:

```
Write-Error "Non ten permiso"
```

```
Write-Error -Exception `
([System.Management.Automation.RuntimeException]"Error
xenérico")
```

```
Write-Error -Exception (New-Object -TypeName `
"System.Management.Automation.RuntimeException") -Message "Outro
erro"
```

```
Write-Error "E este error é fatal" -ErrorAction "Stop"
```

A execución dá o seguinte resultado:

```
PS C:\> .\script.ps1
```

```
C:\script1.ps1 : Non ten permiso
```

```
    + CategoryInfo          : NotSpecified: (:) [Write-Error],
WriteErrorException
```

```
    + FullyQualifiedErrorId :
Microsoft.PowerShell.Commands.WriteErrorException,script1.ps1
```

```
C:\script1.ps1 : Error xenérico
```

```
    + CategoryInfo          : NotSpecified: (:) [Write-Error],
RuntimeException
```

```
    + FullyQualifiedErrorId :
System.Management.Automation.RuntimeException,script1.ps1
```

```
C:\script1.ps1 : Outro erro
```

```
    + CategoryInfo          : NotSpecified: (:) [Write-Error],
RuntimeException
```

```
    + FullyQualifiedErrorId :
System.Management.Automation.RuntimeException,script1.ps1
```

```
Este erro é fatal
```

```

+ CategoryInfo           : NotSpecified: (:) [Write-Error],
WriteErrorException

+ FullyQualifiedErrorId   :
Microsoft.PowerShell.Commands.WriteErrorException,script1.ps1

PS C:\>

```

Por certo, se capturamos unha excepción nunha cláusula “catch”, podemos volver a lanzala con “throw”, e incluso poderíamos lanzar unha excepción distinta. Se eliximos esta última opción, deberíamos encapsular a excepción antiga (\$_.Exception) dentro da nova para manter a información orixinal. Porén, a traza do log dirá que o fallo está na liña do “throw”, porque a nova excepción está a ocorrer nese punto.

Se queremos lanzar un erro enteiro (non só a excepción), podemos usar \$PSCmdlet.ThrowTerminatingError(). Neste caso, pódese encapsular o error orixinal pasando \$_. A vantaxe de utilizar este mecanismo é que mantén o punto onde o erro ocorreu orixinalmente.

Vexamos un exemplo:

```

try {
    Write-Host "Procesando ficheiro"

    $content = Get-Content -Path "C:\Noexiste.txt" -ErrorAction
    "Stop"

    Write-Host "Ficheiro procesado"
} catch
[System.Management.Automation.ActionPreferenceStopException] {
    throw (New-Object -TypeName "System.IO.FileNotFoundException"
    -ArgumentList "Ficheiro non atopado", $_.Exception)
} finally {
    Write-Host "Finally:" $error[0].Exception.Message

    Write-Host "Finally:"
    $error[0].Exception.InnerException.Message
}

```

A execución dá o seguinte resultado:

```

PS C:\> .\script.ps1

Procesando ficheiro

Finally: Ficheiro non atopado

```

Finally: No se encuentra la ruta de acceso 'C:\Noexiste.txt' porque no existe.

Ficheiro non atopado

En C:\script1.ps1: 6 Carácter: 8

```
+                throw <<<<                (New-Object -TypeName
"System.IO.FileNotFoundException" `

                + CategoryInfo                : OperationStopped: (:) [],
FileNotFoundException

                + FullyQualifiedErrorId : Fichero no encontrado
```

PS C:\>

Referencias:

[1]. Kevin Marquette. "PowerShell Exceptions: Everything You Ever Wanted to Know". En liña: <https://kevinmarquette.github.io/2017-04-10-Powershell-exceptions-everything-you-ever-wanted-to-know/>. Accedido o 19-09-2019.

1.36 Trampas

En PowerShell, unha trampa é un trozo de código que se executará cada vez que ocorra unha excepción no entorno no que se declarara a trampa. Por exemplo, se se declara nunha función, se executará cada vez que ocorra unha excepción nesa función. Despois de executarse o código de la trampa, continuará executando el código normalmente. A súa estrutura é:

```
trap {<COMANDOS>}
```

Por exemplo:

```
function probartrampa {

trap {

    Write-Host $_.Exception.Message $_.Exception.GetType().Name

}

throw [System.Exception]"Primeira"

throw [System.Exception]"Segunda"

throw [System.Exception]"Terceira"

}

probartrampa
```

A execución devolve o seguinte resultado:

PS C:\> .\script.ps1

Primeira Exception

...

Segunda Exception

...

Terceira Exception

...

1.37 Depuración

Aínda que fagamos un bo manexo das excepcións, sempre é posible que o noso script teña algún problema difícil de localizar. En PowerShell, existen algúns cmdlets para facer unha depuración (debug) máis efectiva.

Aqueles aos que lles gusta depurar o código a base de escribir os valores das variables e outras cousas tipo “pasei por aquí” na pantalla, tamén teñen alternativas a *Write-Host* como:

- *Write-Debug*: Escribe unha mensaxe de depuración na consola, sempre e cando o modo depuración estea habilitado, xa que por defecto non o está. Este modo contrólase na variable automática *\$DebugPreference* que pode tomar o valor “Continue”, para que PowerShell amose a mensaxe de depuración “SilentlyContinue”, para que siga sen amosala, “Stop”, para que a amose e deteña o script, e “Inquire”, para que lle pregunte ao usuario que facer [1].
- *Write-Verbose*: Escribe mensaxes só se habilitamos o modo detallado, que por defecto está deshabilitado. Este modo se controla na variable automática *\$VerbosePreference*, que pode tomar o valor “Continue”, para que PowerShell amose a mensaxe de depuración “SilentlyContinue”, para que siga sen amosala, “Stop”, para que a amose e deteña o script, e “Inquire”, para que lle pregunte ao usuario que facer [2].

Outro cmdlet de depuración é *Set-PsDebug*, que activa ou desactiva as características de depuración de scripts [3]. Este comando serve, entre outras cousas, para alternar entre o modo de execución estrito e o laxo mediante a opción *-Strict*. En modo estrito, no se poderán facer accións perigosas como empregar variables non declaradas previamente, etc, mentres que en modo laxo, o intérprete de PowerShell resolverá estes problemas como boamente poida.

```
Set-PSDebug -Strict
```

Outra función deste cmdlet é establecer o nivel de seguimento coa opción *-Trace*. Esta pode tomar os valores 0 (sen traza), 1 (facer unha traza de cada unha das liñas de código executadas) ou 2 (facer todo o do 1 e ademais amosar asignacións de variables e chamadas a funcións). Un exemplo para poñer o sistema en nivel 1 sería:

```
Set-PSDebug -Trace 1
```

Ademais, este cmdlet tamén serve para facer que o sistema execute as liñas unha a unha. Isto faise mediante a opción *-Step*.

```
Set-PSDebug -Step
```

Por último, o modo de depuración desactívase mediante a opción -Off, que é incompatible coas outras

```
Set-PSDebug -Off
```

Por último, pero non menos importante, temos o cmdlet *Trace-Command*, que configura e inicia un seguimento da expresión ou do comando especificados [4]. O cmdlet emprega 3 parámetros: -Name, para indicar qué compoñentes de PowerShell van ser avaliados, -Expression, coa expresión que se vai a estudar, e -PSHost, un flag que indica que as trazas deben escribirse na consola de PowerShell.

Por exemplo, supoñamos que queremos ver como o sistema pasa a saída dun comando á entrada doutro a través do pipeline. Un exemplo sería o de listar os directorios mediante *Get-ChildItem*, coller o primeiro resultado e pasarllo a *Move-Item* para que o mande a outro lugar. Unha aproximación sería:

```
Get-ChildItem | Select -first 1 | Move-Item C:\Temp -WhatIf
```

Mais este comando está mal, porque falta o nome do parámetro "-Destination" no último cmdlet, de forma que Windows non sabe se "C:\Temp" é o orixe ou o destino. Entón probamos a trazar o compoñente "ParameterBinding" no comando:

```
Trace-Command -Name ParameterBinding -Expression {Get-ChildItem  
| Select -first 1 | Move-Item C:\Temp -WhatIf} -PSHost
```

E o resultado axudaríanos a ver cal é o problema (en este caso, que a primeira ruta se está asociando ao parámetro -Path e por tanto non hai un valor para o parámetro -Destination).

Referencias:

[1] SS64.com. "Write-Debug". En liña: <https://ss64.com/ps/write-debug.html>. Accedido o 19-09-2019.

[2] Dr. Scripto. "Use PowerShell to Write Verbose Output". En liña: <https://devblogs.microsoft.com/scripting/use-powershell-to-write-verbose-output/>. Accedido o 19-09-2019.

[3] Dr. Scripto. "Troubleshoot by using Set-PSDebug". En liña: <https://devblogs.microsoft.com/scripting/troubleshoot-by-using-set-psdebug/>. Accedido o 19-09-2019.

[4] Dr. Scripto. "Trace Your Commands by Using Trace-Command". En liña: <https://devblogs.microsoft.com/scripting/trace-your-commands-by-using-trace-command/>. Accedido o 19-09-2019.

1.38 Ámbitos

Deixamos este tema para o final porque é bastante complicado. Como nunha linguaxe de programación, as variables en PowerShell teñen ámbitos que limitan en que lugares se pode

ler ou modificar unha variable, aínda que no caso de PS, estes funcionan de xeito lixeiramente distinto, o que pode dar lugar a moitos problemas.

Para comezar, PowerShell crea automaticamente varios ámbitos: o “global”, o de “script” e o “local”. O primeiro entra en vigor en canto iniciamos unha sesión en PowerShell, e o segundo cando se executa un arquivo de script. Porén, o terceiro non funciona como as variables locais das linguaxes de programación, senón que é unha especie de punteiro ao ámbito actual, que pode ser o global, o de script, etc. Ademais, PowerShell tamén crea ámbitos específicos para as funcións, e estes tamén poden ser accedidos a través do ámbito local.

En todo caso, podemos ver as variables de cada ámbito co parámetro “scope” de Get-Variable:

```
PS C:\> get-variable -scope global
```

```
PS C:\> get-variable -scope local
```

E ao acceder a unha variable, pódese especificar un ámbito poñendo o nome do ámbito e o carácter “:” entre o símbolo do dólar e o nome da variable. Por exemplo:

```
$global:b = 'Ola, Script!'
```

```
Write-Host $global:b
```

```
Write-Host $script:b
```

```
Write-Host $local:b
```

E o mesmo é aplicable a funcións, poñendo o ámbito diante do seu nome. Por exemplo, para especificar que unha función se ten que crear no ámbito global, facemos:

```
function global:Hello {  
    Write-Host "Ola, función"  
}
```

Os ámbitos, ademais, poden aniñarse. Un elemento (variable, alias, etc.) é visible tanto no ámbito que se creou como nos sub-ámbitos que se vaian creando dentro dese ámbito, a menos que dito elemento sexa declarado co modificador *private*. Mais os elementos non se herdan. Se temos unha variable \$b nunha liña de comandos podemos vela desde dentro dun script executado nesa mesma liña de comandos, pero iso non significa que a variable esta no ámbito do script ou no local. Así, se temos o seguinte script

```
Write-Host $b
```

```
Write-Host $global:b
```

```
Write-Host $script:b
```

```
Write-Host $local:b
```

E o invocamos, teremos a seguinte saída, cos dúas últimas liñas devolvendo baleiro:

```
PS C:\> .\ambito.ps1
```

```
Ola, variable!
```

```
Ola, variable!
```

```
PS C:\>
```

Agora facemos o seguinte script:

```
for ($i=0; $i -lt 3; $i++) {  
    Write-Host $numero  
    $numero++  
}
```

E na liña de comandos establecemos \$numero, invocamos e volvemos a ver o valor de \$numero. O resultado é:

```
PS C:\Users\admin> $numero=5
```

```
PS C:\Users\admin> .\ambito.ps1
```

```
5
```

```
6
```

```
7
```

```
PS C:\Users\admin> Write-Host $numero
```

```
5
```

É dicir, ao volver ao ámbito global, a variable vale 5 como era de esperar nunha linguaxe común, pero non en PS. Xa que:

- Se dentro do script o ámbito era “script”, e non se herda nada de “global”, como é que \$numero non comeza valendo null en vez de 5?
- Se con \$numero estabamos accedendo á variable “global”, como é que non está modificada ao rematar o script?

A resposta é que ao facer \$numero++ internamente PowerShell está a crear unha variable no ámbito de script, e a está inicializando co mesmo valor que a do ámbito global. Pola súa parte, Write-Host \$numero, busca primeiro no ámbito script e logo no ámbito global, atopando o “global” na primeira iteración e o “script” creado por \$numero++ nas seguintes. Todo isto pódese comprobar facendo get-variable -scope script ao comezo e ao remate do script e entre o Write-Host \$numero e o \$numero++. Ademais, pódese ver a diferenza entre facer \$numero+

+ e `$global:numero++`. Esta última non vai crear a variable no ámbito script e vai ter modificado o valor de `$numero` na consola ao rematar. Maxia.

E aquí vén outro punto con bastante controversia: segundo a documentación de PowerShell, “private” non é máis ca un modificador de visibilidade, non é un ámbito [1], aínda que en moitos documentos podemos ver que falan do ámbito “private” [2] e se facemos o seguinte, funciona correctamente:

```
PS C:\> get-variable -scope private
```

```
PS C:\> Write-Host $private:b
```

En calquera caso, se na liña de comandos declaramos a variable como privada, non se verá no script:

```
PS C:\> New-Variable -Name c -Value 'ola, privada!' -Option private
```

```
PS C:\> Get-Content .\ambito.ps1
```

```
Write-Host $global:c
```

```
PS C:\> .\ambito.ps1
```

```
PS C:\>
```

De forma similar ás linguaxes de programación a variable do ámbito externo escurécese fronte á do ámbito máis interno. No seguinte exemplo, vemos como a variable `$a` dentro do script `ambito.ps1` non é a mesma que a variable `$a` da liña de comandos:

```
PS C:\> Get-Content ambito.ps1
```

```
$a='Ola, Script!'
```

```
Write-Host $a
```

```
PS C:\> $a = 'ola, PS!'
```

```
PS C:\> Write-Host $a
```

```
ola, PS!
```

```
PS C:\> .\ambito.ps1
```

```
Ola, Script!
```

```
PS C:\> Write-Host $a
```

```
ola, PS!
```

Para acceder á variable `$a` do ámbito global desde dentro do script, teríamos que ter empregado `$global:a` onde fora conveniente.

Co obxectivo de liar isto aínda máis, Microsoft creou unha opción chamada AllScope, que ao estar activada (-Option AllScope ao crear a variable) fai que a variables si se herde no sub-ámbito de turno, pero non se vai herdar retroactivamente no ámbito pai.

E o non va máis é que a partir de PS 3.0, tamén temos a posibilidade de empregar variables locais en comandos remotos, traballos en segundo plano, threads, etc. Para iso, temos que empregar o modificador "Using:". Por exemplo:

```
$ps = "*PowerShell*"

Invoke-Command -ComputerName S1 -ScriptBlock {

    Get-WinEvent -LogName $Using:ps

}
```

Referencias:

- [1] Microsoft: "Acerca de los ámbitos" En liña: https://docs.microsoft.com/es-es/powershell/module/microsoft.powershell.core/about/about_scopes. Accedido o 04-02-2021.
- [2] ss64.com. "How-to: Define the Scope of a variable". En liña: <https://ss64.com/ps/syntax-scopes.html>. Accedido o 04-02-2021.